Lab 02

Hands-On Git and Version Control

Name: Saba Gull

Registration No: 2023-BSCS-074

Course Code: CS-649 (Cloud DevOps)

Section: A

GitHub Username: sabagu003-ui

Getting Started: Install Git

Step 1: Installed Git on my Ubuntu machine and checked the installed version.

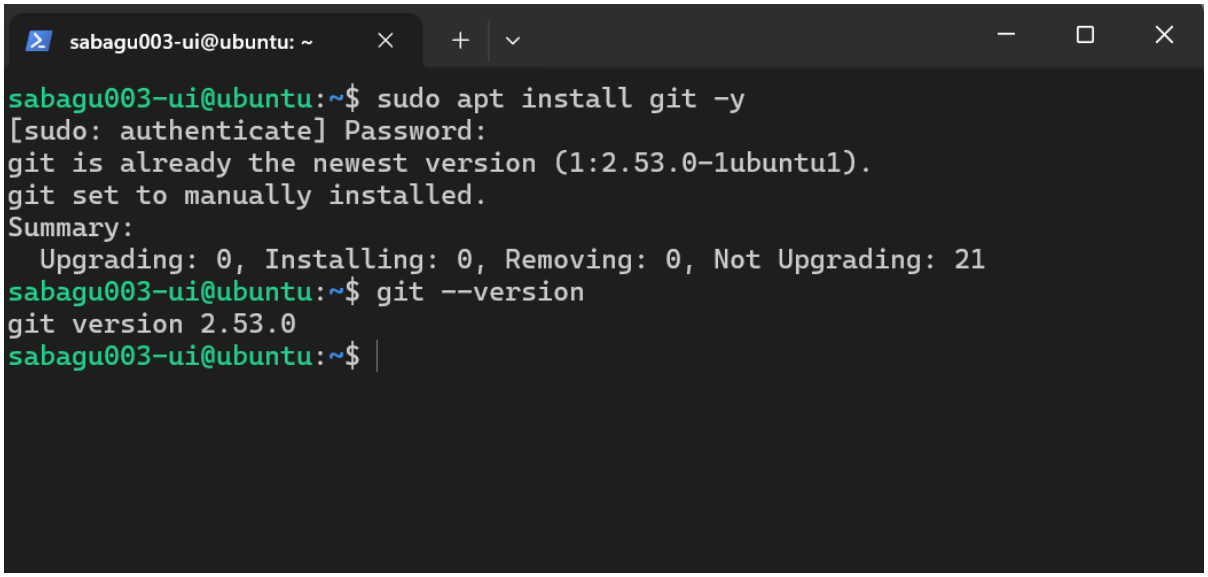


Figure 0.1: Git installed; git --version shows version 2.53.0. (git_installation.png)

Task 1: Create Private GitHub Repository

Step 1: Opened GitHub > New repository, entered the name DevOps_Lab2_Practice and selected the Private visibility option.

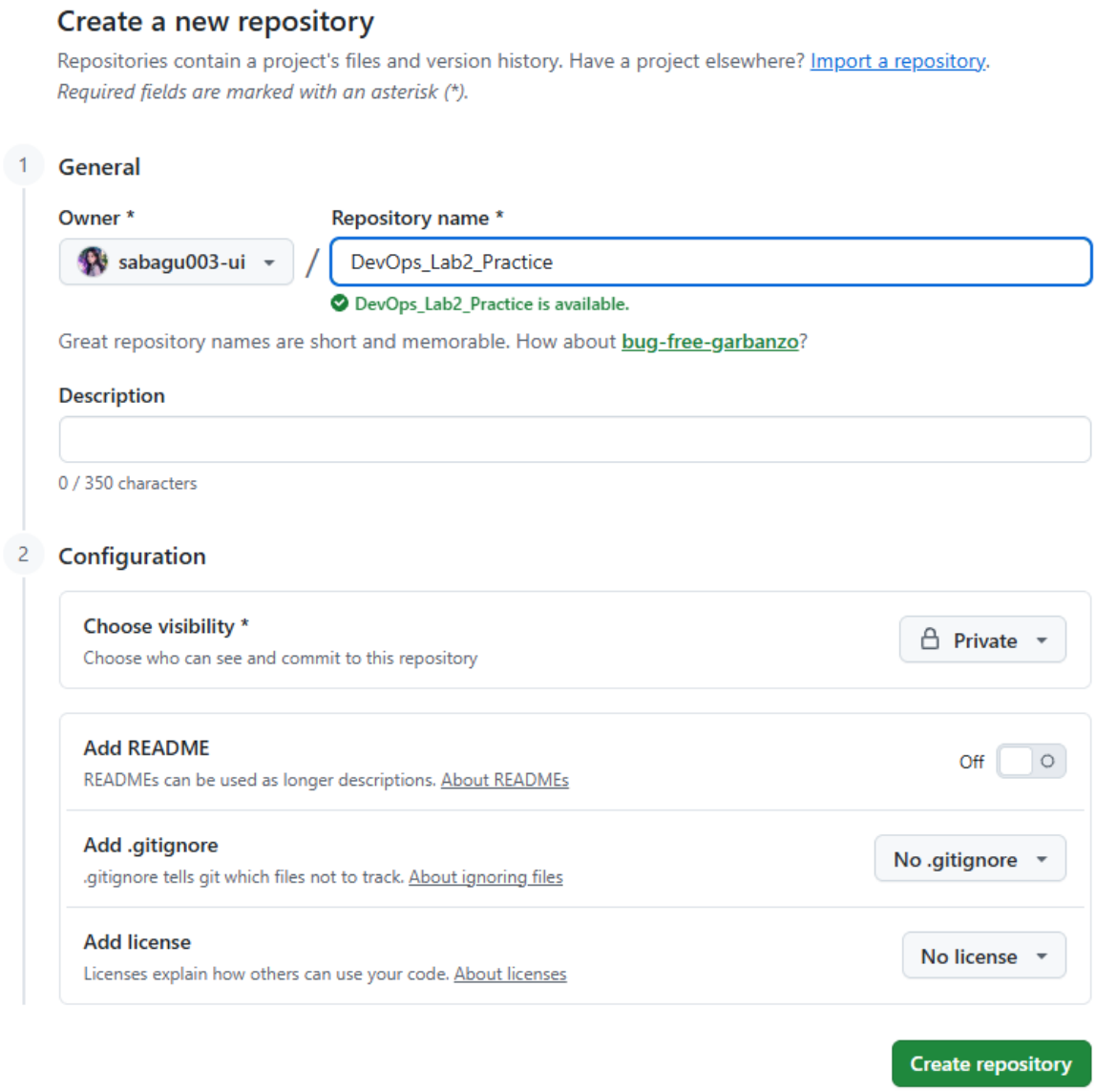


Figure 1.1: Create a new repository page with the name DevOps_Lab2_Practice and Private selected.

Step 2: Created the repository and checked on GitHub that it is shown as Private, next to my Ubuntu terminal.

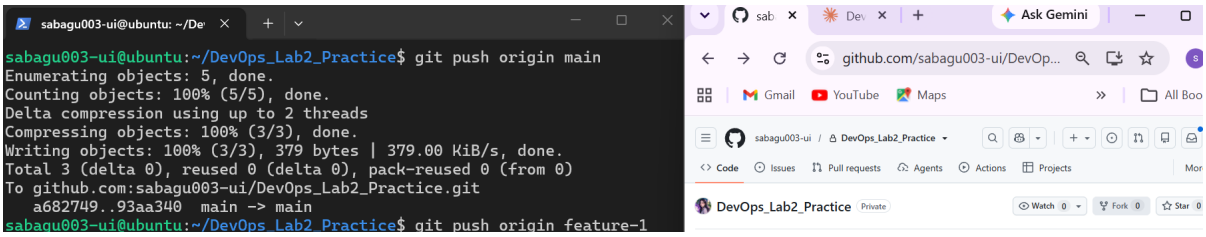


Figure 1.2: Split-screen: DevOps_Lab2_Practice shows the Private badge and the terminal prompt shows sabagu003-ui@ubuntu. (repo_private.png)

Task 2: Connect Repository via SSH

Step 1: Generated a new ed25519 SSH key and displayed the public key.

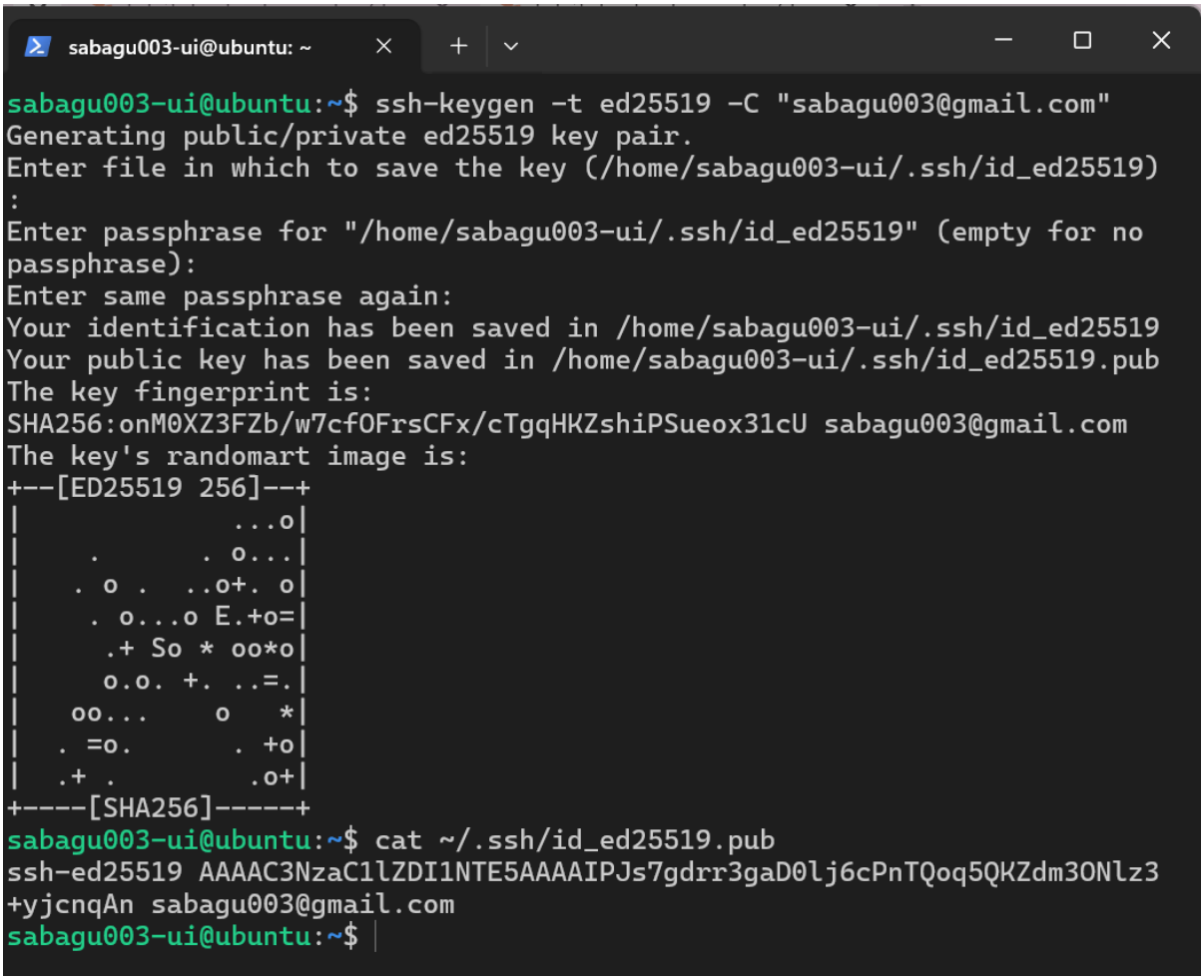


Figure 2.1: ssh-keygen output and the public key (cat ~/.ssh/id_ed25519.pub). The private key is not shown. (ssh_keygen.png)

Step 2: Opened GitHub > Settings > SSH and GPG keys > New SSH key and pasted the public key with the title Ubuntu VM.

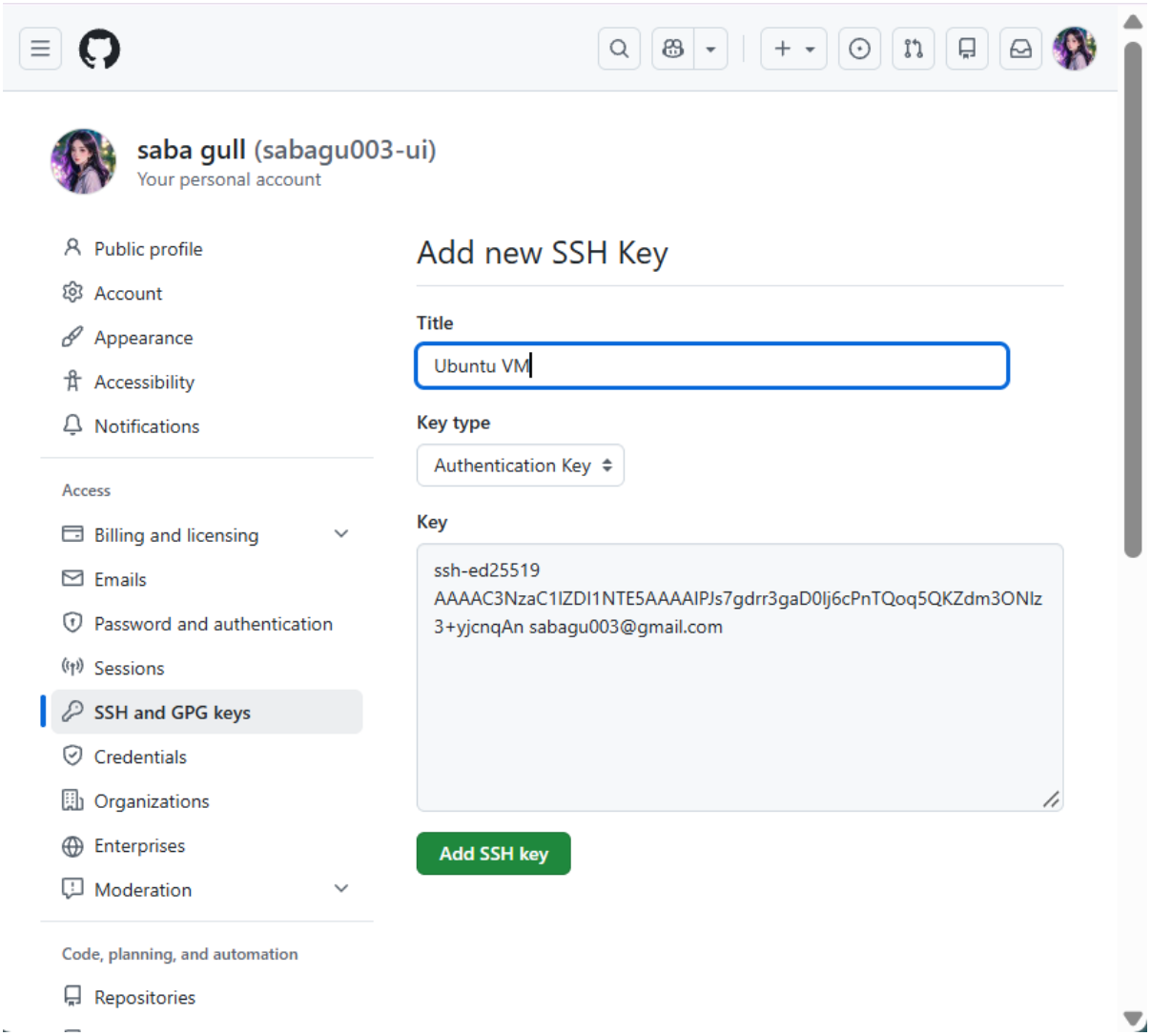


Figure 2.2: Add new SSH key form on GitHub with title Ubuntu VM.

Step 3: Added the key; GitHub confirmed that the key was added successfully.

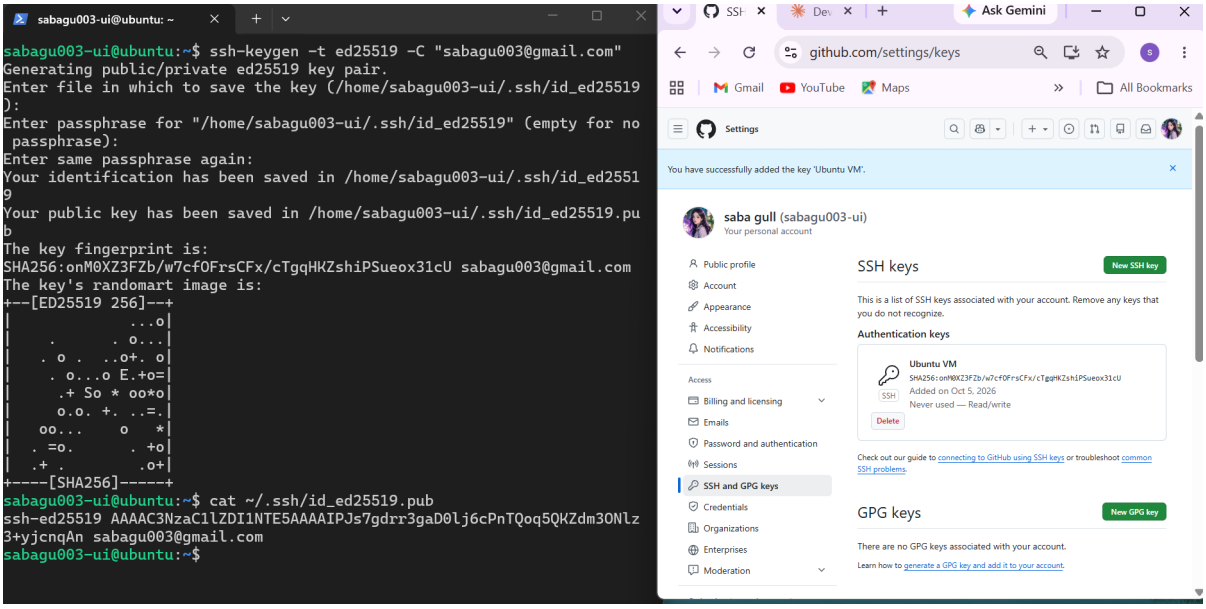


Figure 2.3: Split-screen: SSH key "Ubuntu VM" added in GitHub settings and the terminal showing sabagu003-ui@ubuntu. (github_sshkey.png)

Step 4: Tested the SSH connection with GitHub and cloned the DevOps_Lab2_Practice repository using SSH.

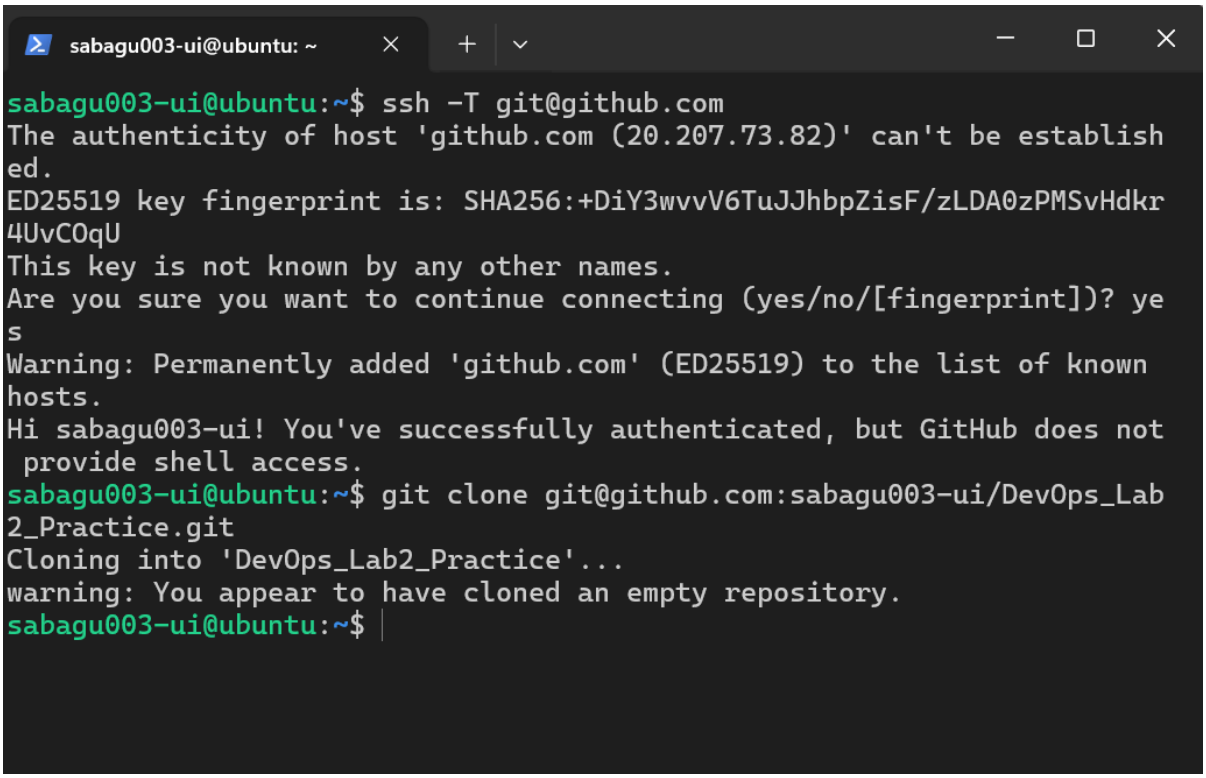


Figure 2.4: ssh -T authenticated successfully and the repository was cloned using SSH. (ssh_clone.png)

Task 3: Configure Git Username and Email

Step 1: Set my Git username and email.

```
sabagu003-ui@ubuntu:~$ git config --global user.name "Saba Gull"
sabagu003-ui@ubuntu:~$ git config --global user.email "sabagu003@gmail.
com"
sabagu003-ui@ubuntu:~$
```

Figure 3.1: git config --global user.name and user.email configured. (git_identity.png)

Step 2: Verified the configuration with git config --list.

```
sabagu003-ui@ubuntu:~$ git config --list
user.name=Saba Gull
user.email=sabagu003@gmail.com
sabagu003-ui@ubuntu:~$
```

Figure 3.2: git config --list showing user.name and user.email. (git_config_list.png)

Task 4: Explore the .git Folder

Step 1: Entered the cloned repository, showed hidden files and listed the contents of the .git folder.

```
sabagu003-ui@ubuntu:~$ cd DevOps_Lab2_Practice
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ ls -a
.  ..  .git
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ ls -a .git
.  ..  HEAD  config  description  hooks  info  objects  refs
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 4.1: Hidden .git folder and its contents (HEAD, config, description, hooks, info, objects, refs). (git_folder.png)

Task 5: Local Repository Management

Step 1: Deleted the .git folder and ran git status. It gives the error "fatal: not a git repository" because all the history, configuration and tracking data of a repository is stored inside the .git folder; without it the folder is just a normal folder.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ rm -rf .git
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git status
fatal: not a git repository (or any of the parent directories): .git
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 5.1: Error after deleting .git: fatal: not a git repository. (delete_git.png)

Step 2: Re-initialized the local Git repository with git init.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git init
hint: Using 'master' as the name for the initial branch. This default b
ranch name
hint: will change to "main" in Git 3.0. To configure the initial branch
 name
hint: to use in all of your new repositories, which will suppress this
warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this co
mmand:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchNam
e false"
Initialized empty Git repository in /home/sabagu003-ui/DevOps_Lab2_Prac
tice/.git/
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 5.2: Empty Git repository initialized again. (git_init.png)

Step 3: Created README.md and made the first commit.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ echo "# Lab02 Git Practice"
 >> README.md
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git add README.md
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Initial comm
it"
[master (root-commit) 2f0935f] Initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 5.3: README.md added and committed with the message Initial commit. (first_commit.png)

Step 4: Added the GitHub remote, renamed the branch to main and pushed it.

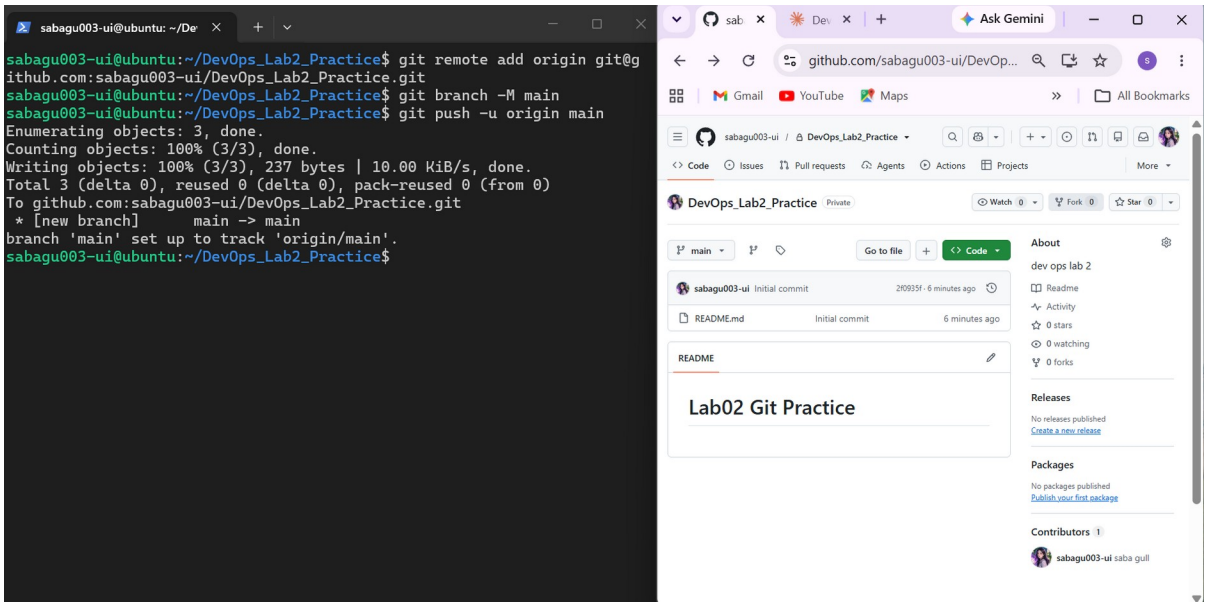


Figure 5.4: Split-screen: main branch pushed to GitHub and README.md visible in the repository. (first_push.png)

Task 6: File Status & Staging

Step 1: Created notes.txt and checked the status; the file is shown as untracked.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ echo "Adding some test note
s" >> notes.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notes.txt

nothing added to commit but untracked files present (use "git add" to t
rack)
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 6.1: git status showing notes.txt as an untracked file. (status1.png)

Step 2: Staged and committed notes.txt.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add notes.tx
t"
[main 3e363c9] Add notes.txt
 1 file changed, 1 insertion(+)
 create mode 100644 notes.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 6.2: notes.txt staged and committed with the message Add notes.txt. (commit_notes.png)

Step 3: Edited notes.txt, checked the status (modified file), then staged and committed it again.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add notes.tx
t"
[main 3e363c9] Add notes.txt
 1 file changed, 1 insertion(+)
 create mode 100644 notes.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ echo "Second note" >> notes
.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   notes.txt

no changes added to commit (use "git add" and/or "git commit -a")
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git add notes.txt
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Update notes
.txt"
[main c6ca757] Update notes.txt
 1 file changed, 1 insertion(+)
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 6.3: Second note added to notes.txt, status checked, then add and commit repeated.

Step 4: Pushed the updated files (notes.txt and main.py) to GitHub.

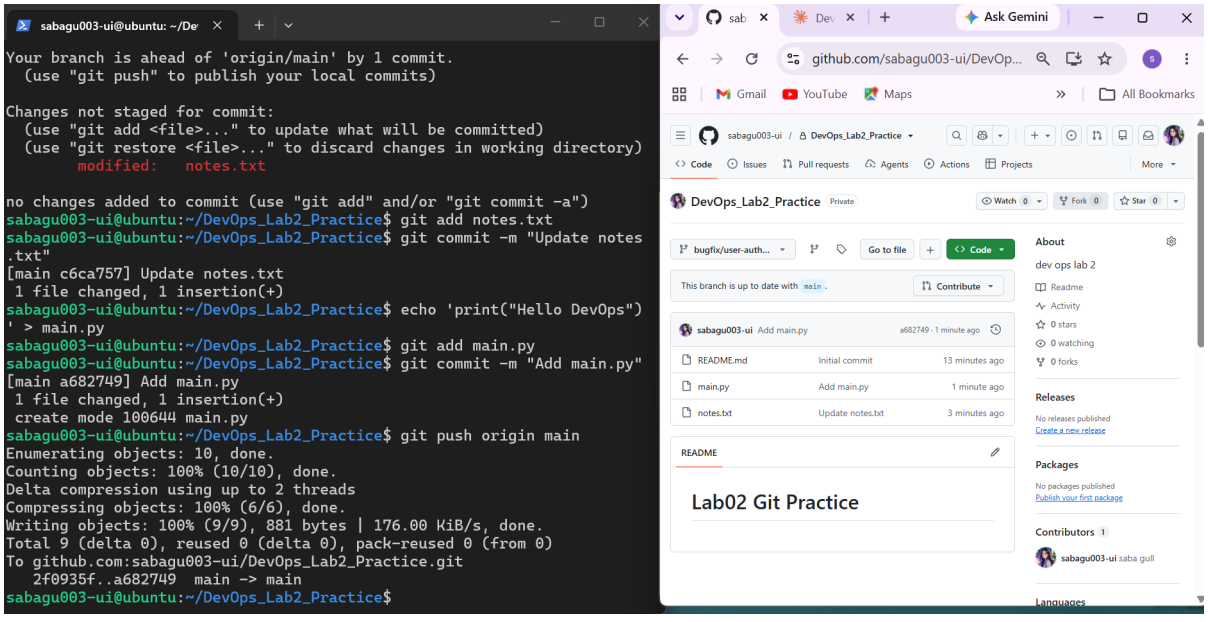


Figure 6.4: Split-screen: notes.txt and main.py committed and pushed to main on GitHub.

Task 7: Branch Creation Using GitHub GUI

Step 1: Created the branch bugfix/user-auth-error from the GitHub web interface (branch dropdown > create branch).

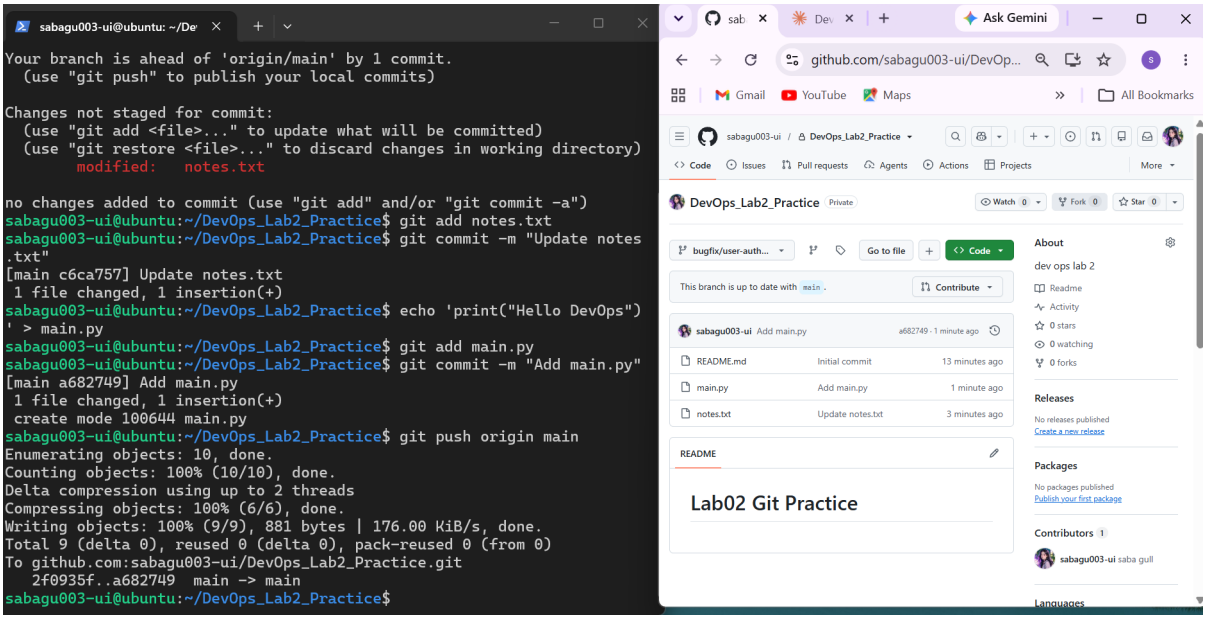


Figure 7.1: Split-screen: branch bugfix/user-auth-error created on GitHub and the terminal with sabagu003-ui@ubuntu. (bugfix_branch_gui.png)

Step 2: Pulled the branch to my local repository, checked it out and listed the local branches.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git pull origin bugfix/user-auth-error
From github.com:sabagu003-ui/DevOps_Lab2_Practice
 * branch            bugfix/user-auth-error -> FETCH_HEAD
Already up to date.
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git fetch origin
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git checkout bugfix/user-auth-error
branch 'bugfix/user-auth-error' set up to track 'origin/bugfix/user-auth-error'.
Switched to a new branch 'bugfix/user-auth-error'
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git branch
* bugfix/user-auth-error
  main
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 7.2: Local git branch list showing bugfix/user-auth-error and main. (bugfix_branch_local.png)

Task 8: Branch Creation and Push Using Git Bash

Step 1: Created the branch feature/db-connection, added db.py, committed it and pushed the branch to the remote repository.

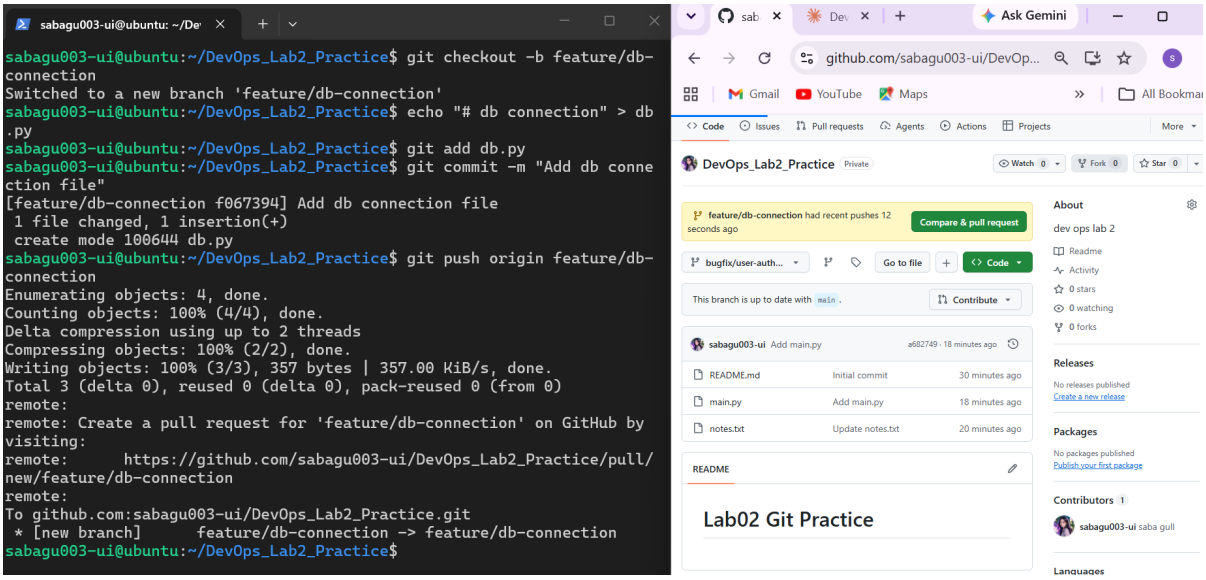


Figure 8.1: Branch feature/db-connection created, committed and pushed to GitHub. (feature_db_branch.png)

Step 2: Switched back to the main branch.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 8.2: Switched back to main.

Task 9: Branching & Merging

Step 1: Created and switched to the new branch feature-1.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git checkout -b feature-1
Switched to a new branch 'feature-1'
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 9.1: Branch feature-1 created. (branch_create.png)

Step 2: Added a function to main.py and committed it.

```
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ echo -e '\ndef greet():\n
  print("Hi from feature-1")' >> main.py
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git add main.py
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$ git commit -m "Add new func
tion to main.py"
[feature-1 93aa340] Add new function to main.py
 1 file changed, 3 insertions(+)
sabagu003-ui@ubuntu:~/DevOps_Lab2_Practice$
```

Figure 9.2: New function added to main.py and committed on feature-1. (feature_commit.png)

Step 3: Switched back to main and merged feature-1 (fast-forward merge).

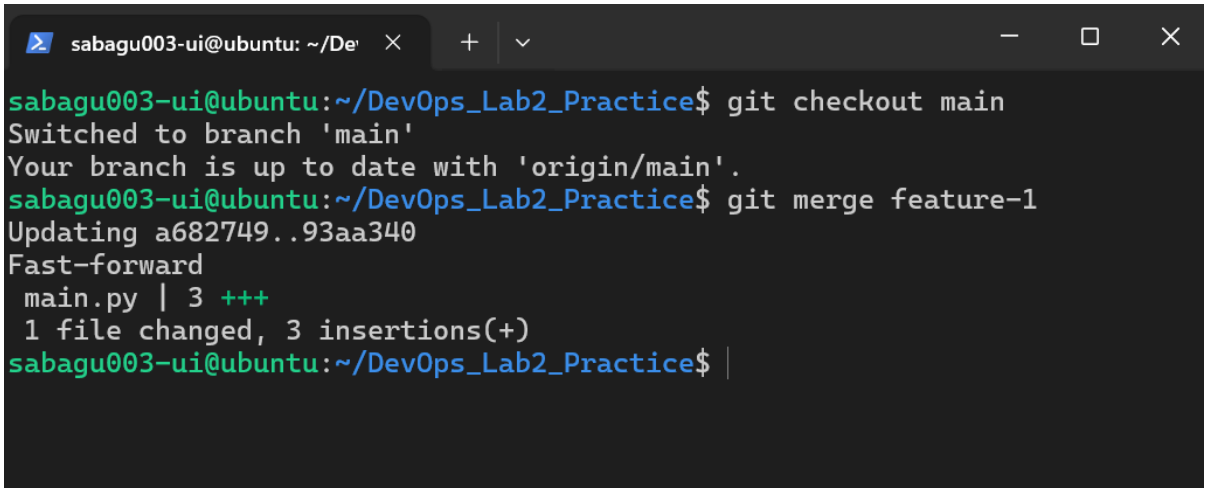


Figure 9.3: feature-1 merged into main. (merge.png)

Step 4: Pushed the main and feature-1 branches to GitHub.

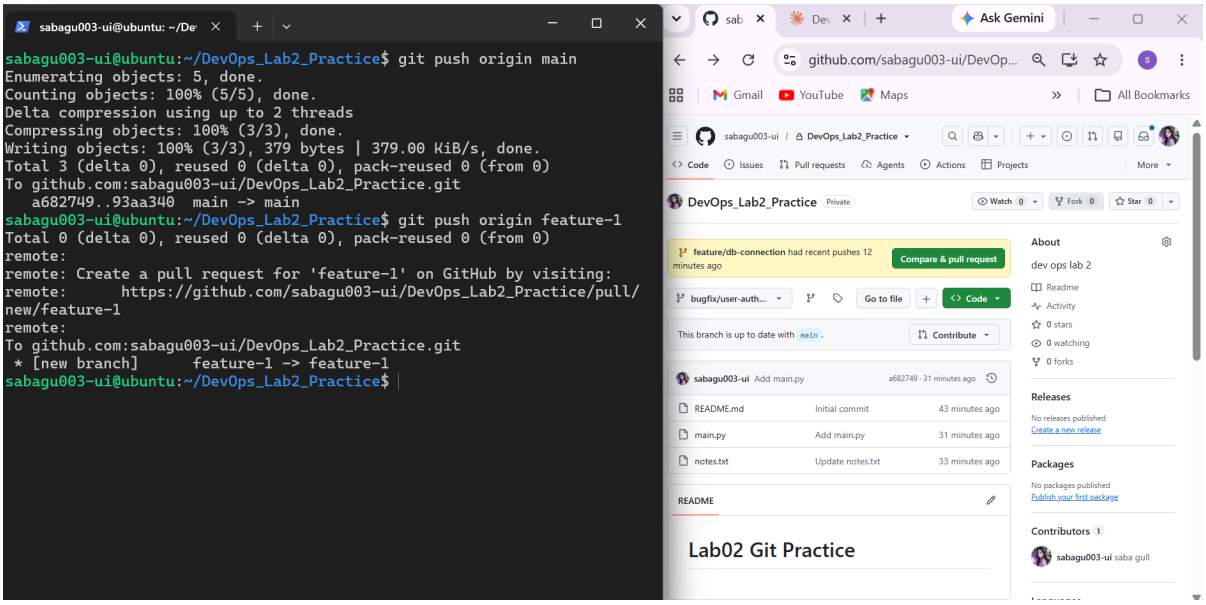


Figure 9.4: Split-screen: main and feature-1 pushed to GitHub. (push_branches.png)

Task 10: Pull Request and Branch Review (GitHub GUI)

Step 1: Created a Pull Request from feature/db-connection to main with a title and a description.

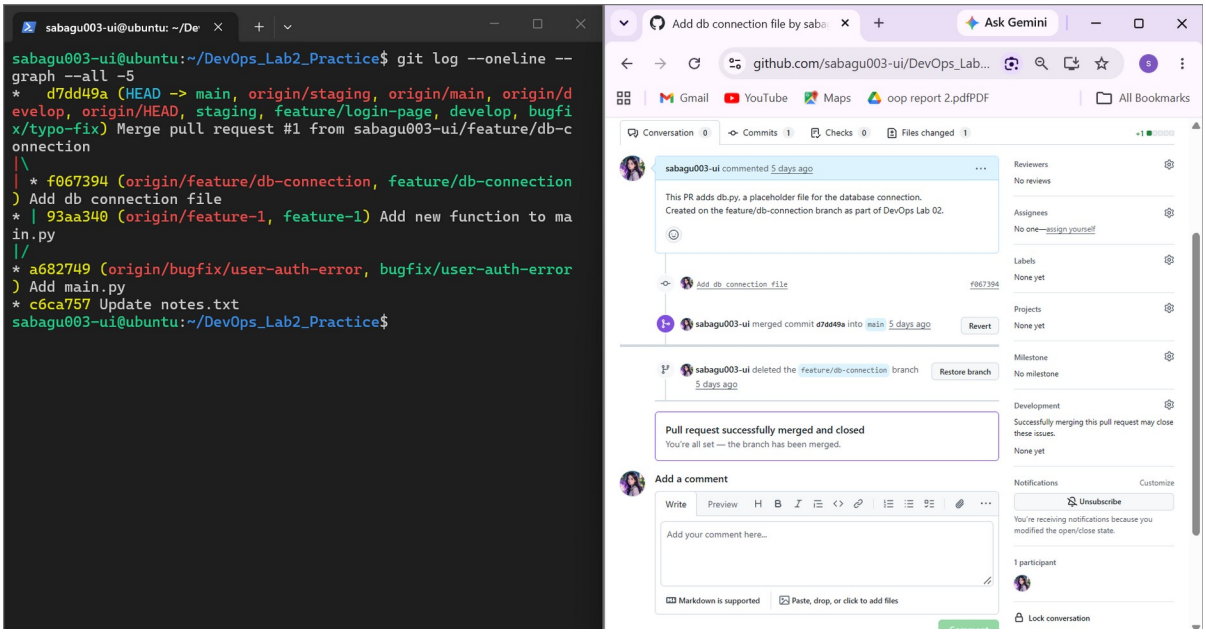


Figure 10.1: Pull Request #1 (Add db connection file) from feature/db-connection to main. (pr_creation.png)

Step 2: Reviewed the Pull Request (Files changed) and merged it using the GitHub GUI.

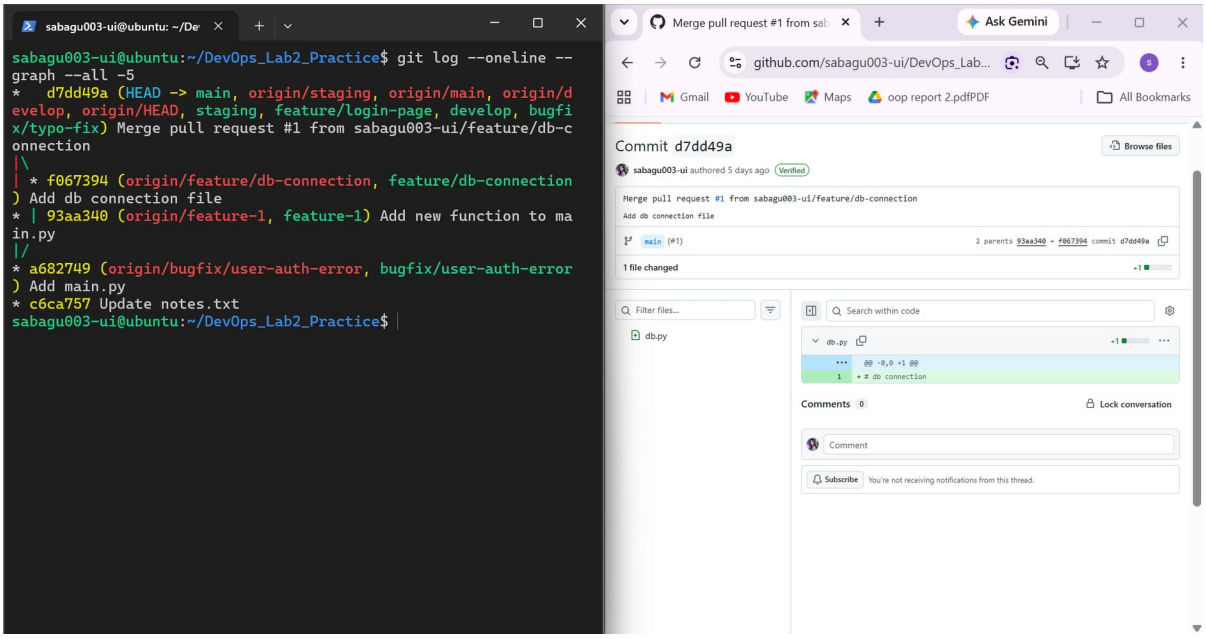


Figure 10.2: Merge pull request step on the Pull Request page. (pr_merge.png)

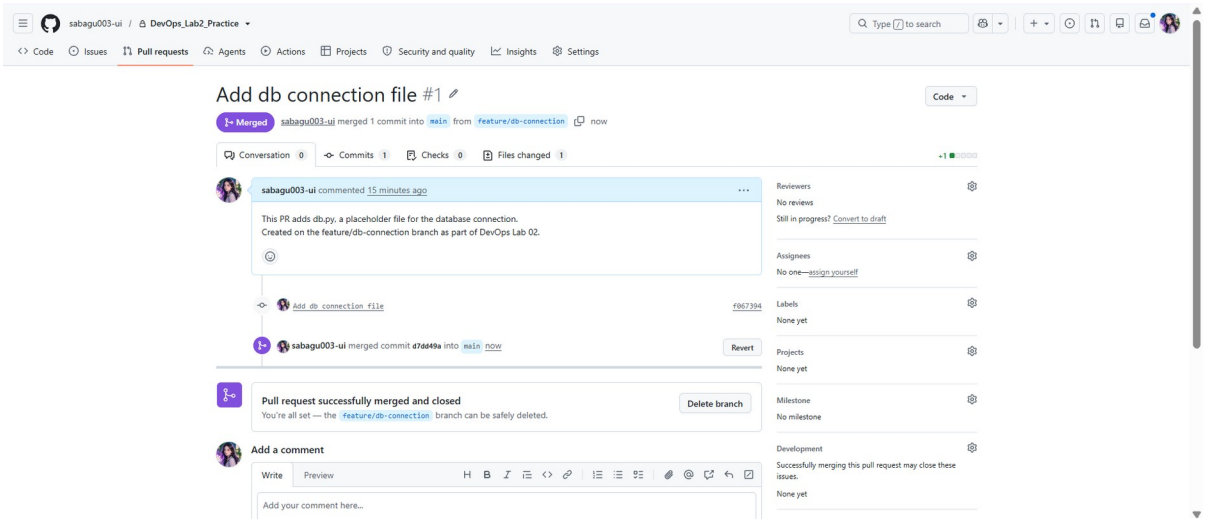


Figure 10.3: Pull Request merged and closed successfully.

Step 3: Deleted the feature/db-connection branch from the GitHub GUI after merging.

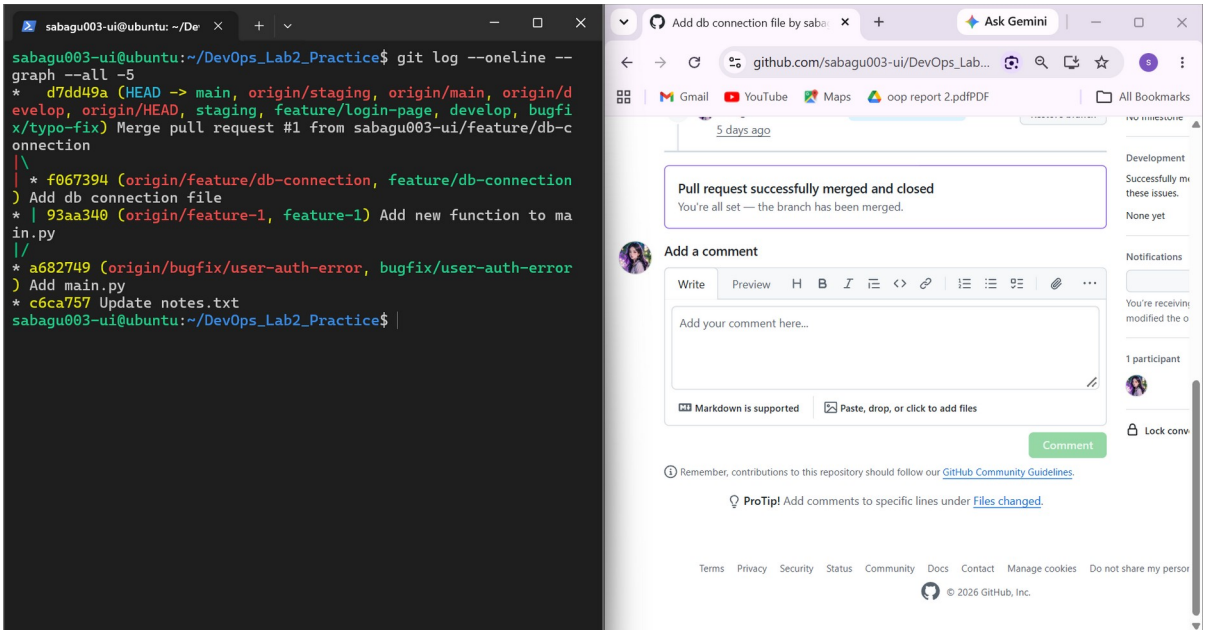


Figure 10.4: Pull Request page showing the feature/db-connection branch deleted after the merge. (branch_delete.png)

Task 11: Detailed Branch Strategy (Develop/Staging)

Step 1: Created the develop, staging, feature/* and bugfix/* branches and listed all branches.

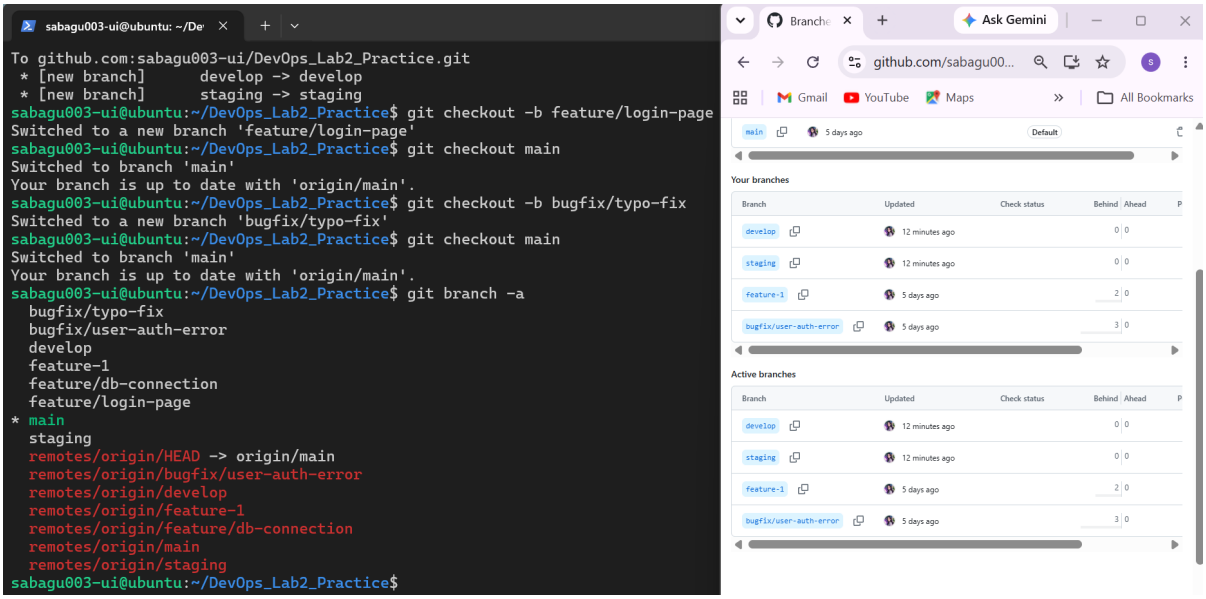


Figure 11.1: Branch list: main, develop, staging, feature/login-page and bugfix/typo-fix. (branch_strategy.png)

Step 2: Committed a change on the feature branch, merged it into develop, then merged develop into staging.

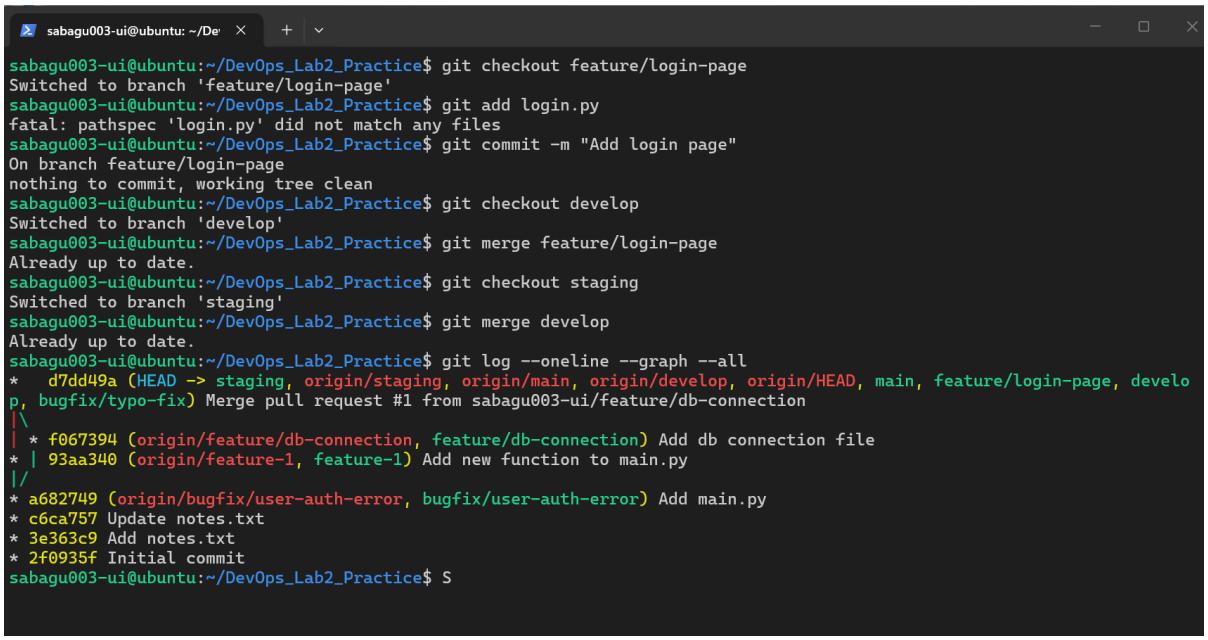


Figure 11.2: Merges into develop and staging. (branch_merges.png)

Step 3: Merged staging into main (production) and pushed all branches.

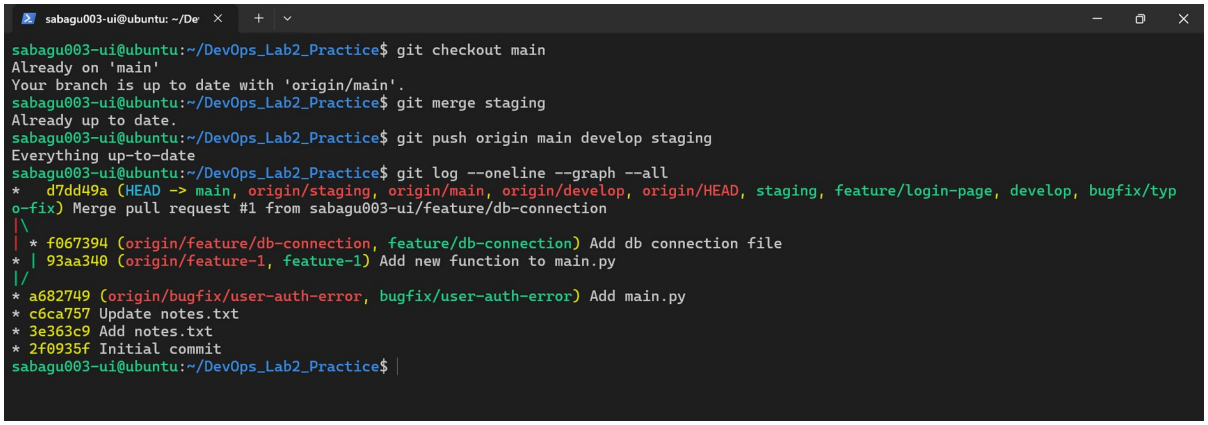


Figure 11.3: Final merge of staging into main. (final_merge.png)

Task 12: Code Review Workflow

Step 1: Created a feature branch, pushed a change and opened a Pull Request from the feature branch into main with a clear title and description.

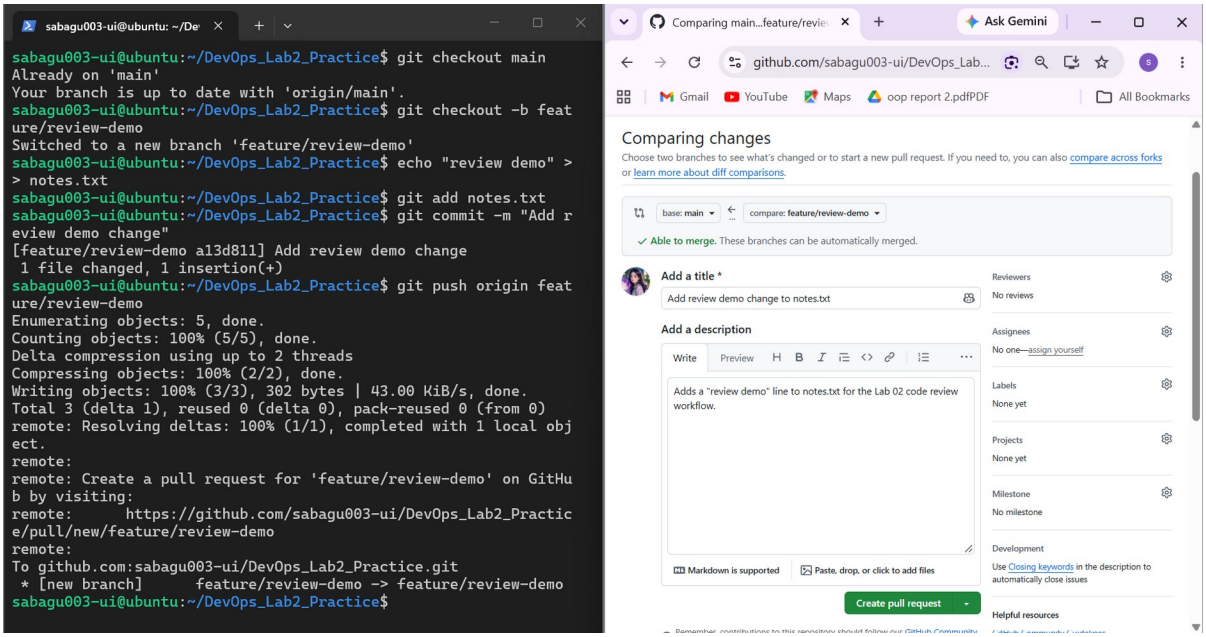


Figure 12.1: Split-screen: Pull Request with title and description. (pr_create_details.png)

Step 2: Assigned a reviewer (second account / teammate) to the Pull Request.

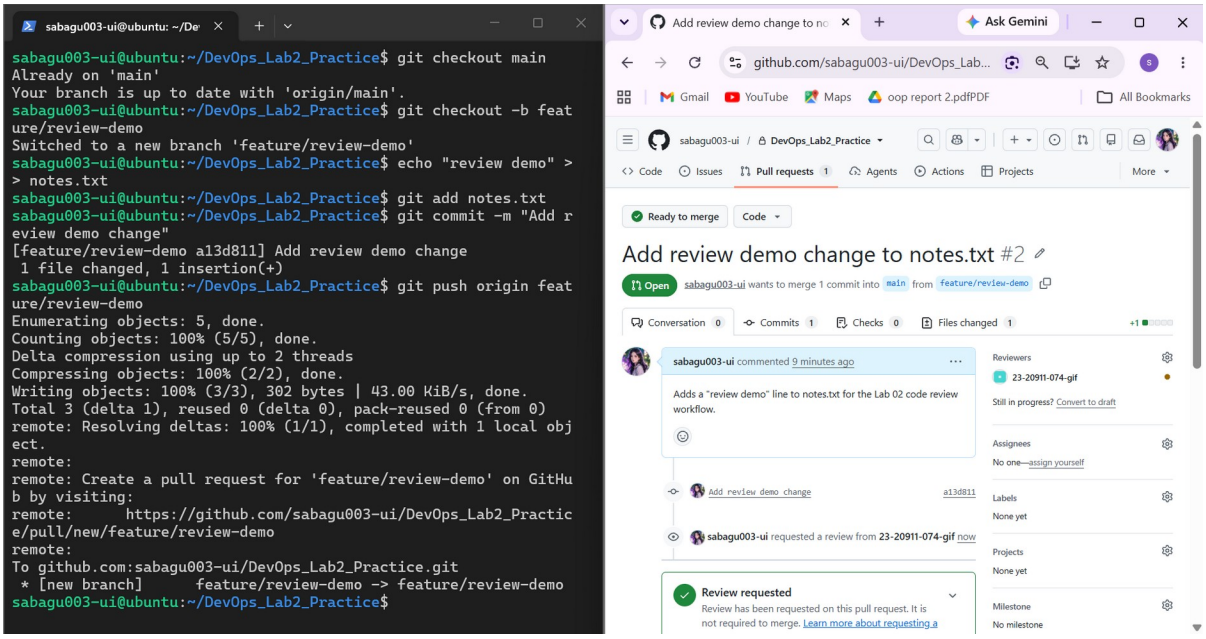


Figure 12.2: Split-screen: reviewer assigned in the Reviewers section. (pr_assigned_reviewer.png)

Step 3: Reviewer requested changes and added comments.

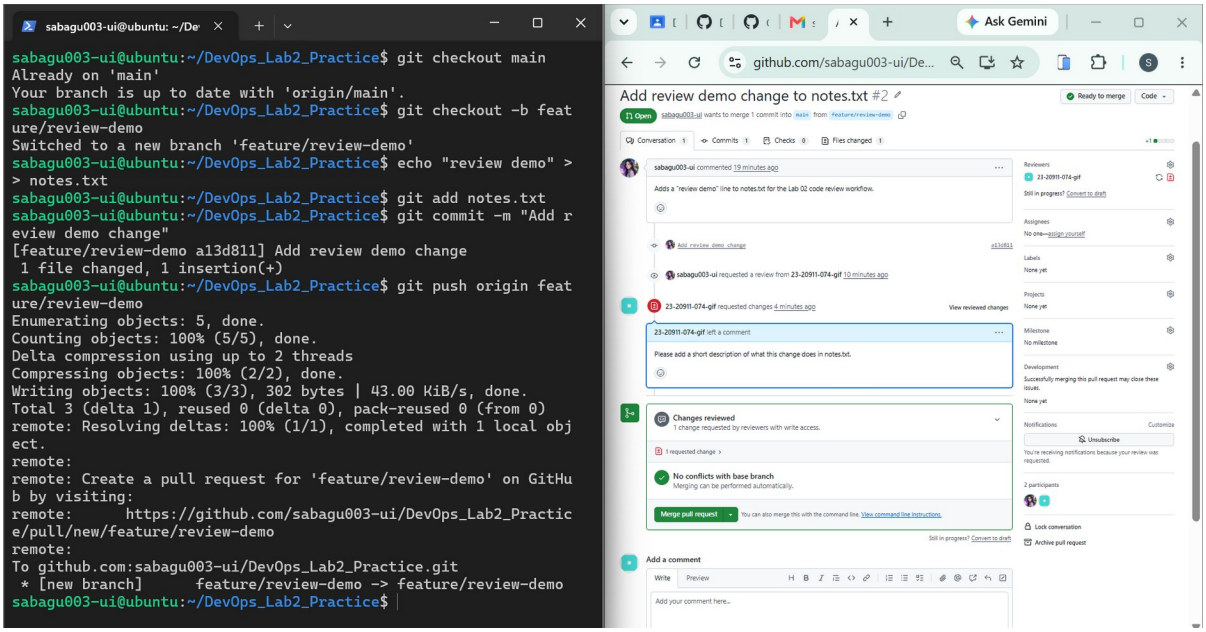


Figure 12.3: Split-screen: review with Request changes and comments. (pr_request_changes.png)

Step 4: Created a separate practice Pull Request and closed it without merging.

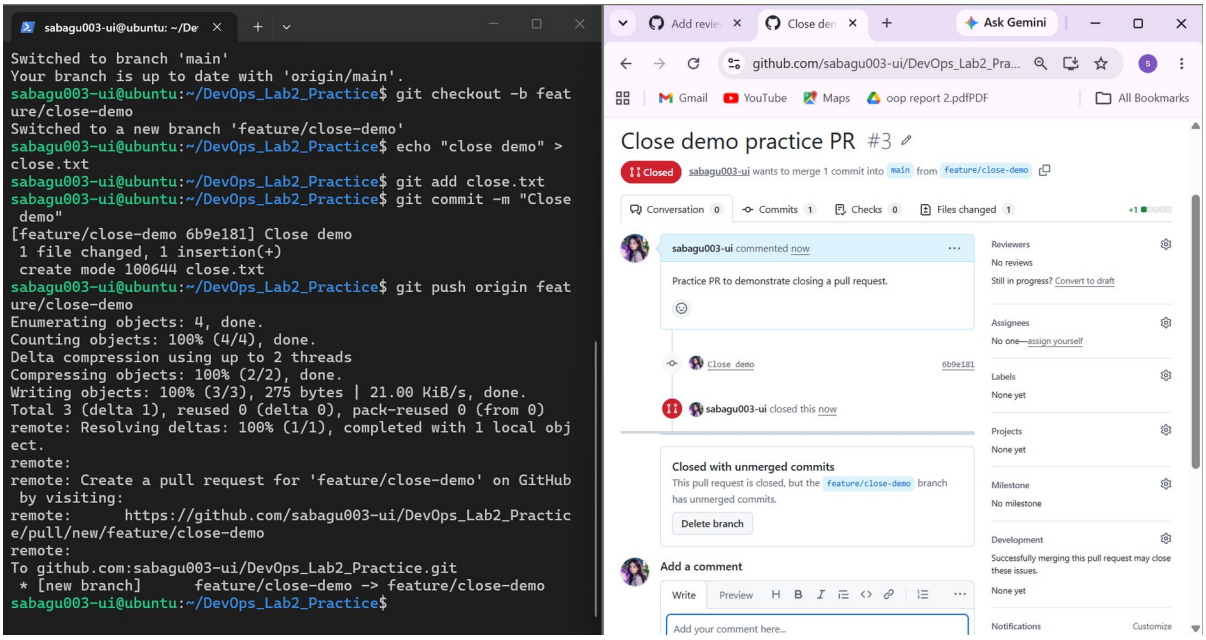


Figure 12.4: Split-screen: practice Pull Request closed. (pr_rejected.png)

Step 5: Author fixed the issues in the same branch and pushed new commits; the Pull Request updated automatically.

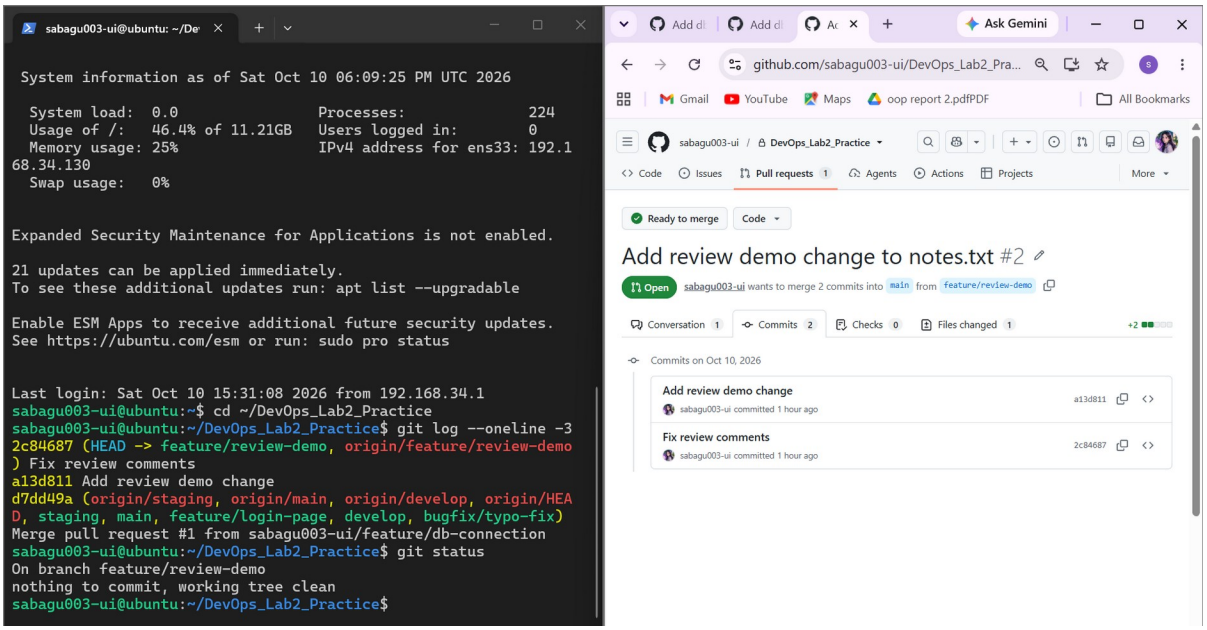


Figure 12.5: Pull Request updated with the new commits. (pr_updated_with_commits.png)

Step 6: Reviewer approved the Pull Request; selected the merge style and opened the merge confirmation.

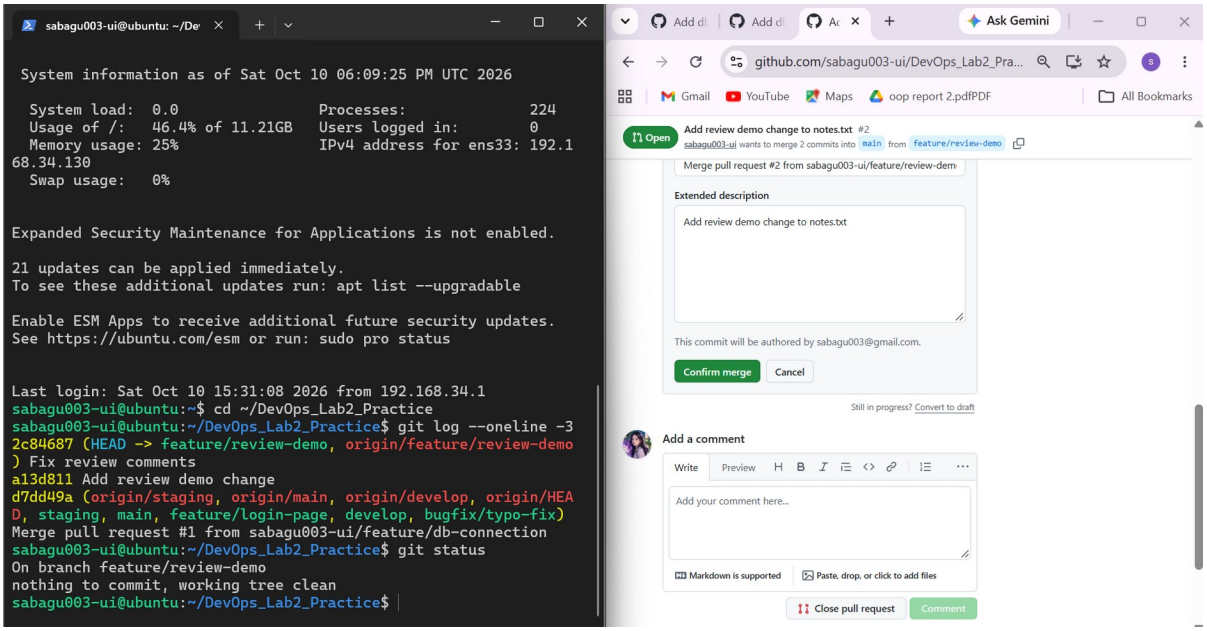


Figure 12.6: Split-screen: merge confirmation. (pr_merge_confirm.png)

Step 7: Checked the merged Pull Request.

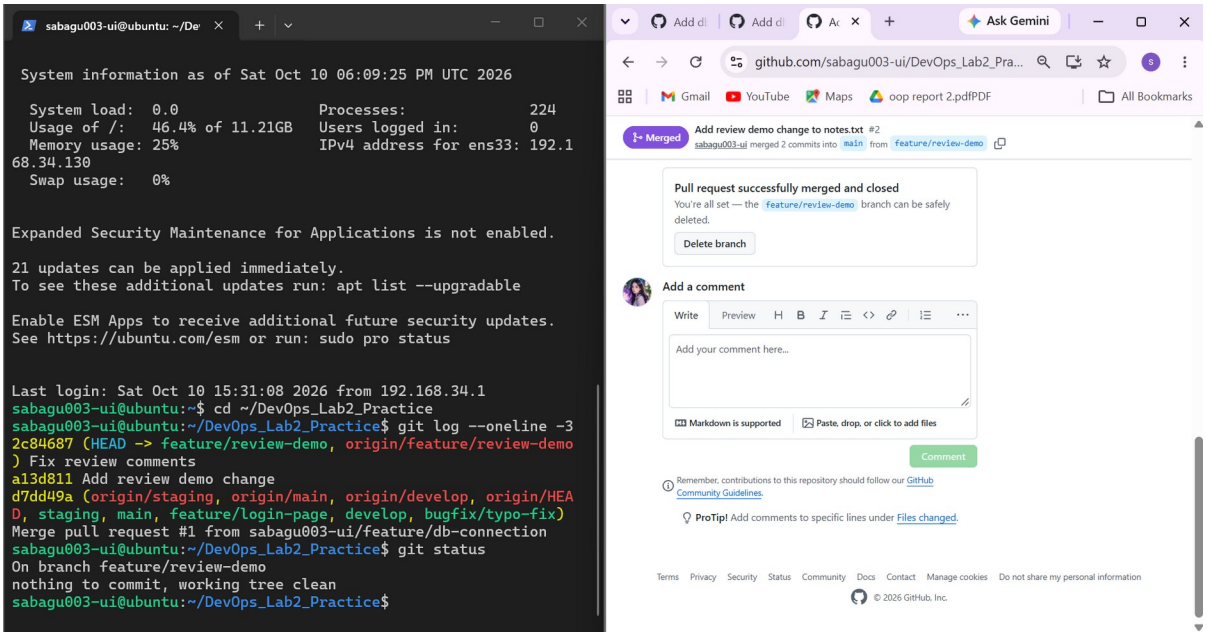


Figure 12.7: Split-screen: Pull Request shown as Merged. (pr_merged.png)

Step 8: Deleted the merged Pull Request branch.

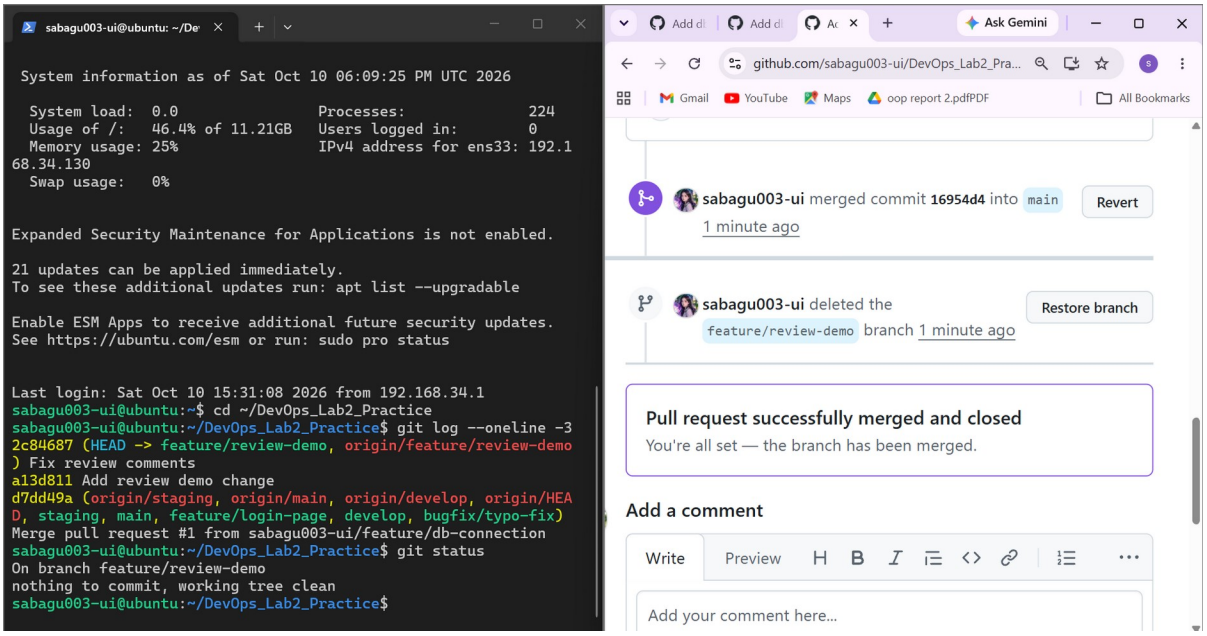


Figure 12.8: Split-screen: Pull Request branch deleted. (pr_branch_deleted.png)

Task 13: Branch Cleanup Best Practices

Step 1: Deleted a merged remote branch using the GitHub UI (Branches page).

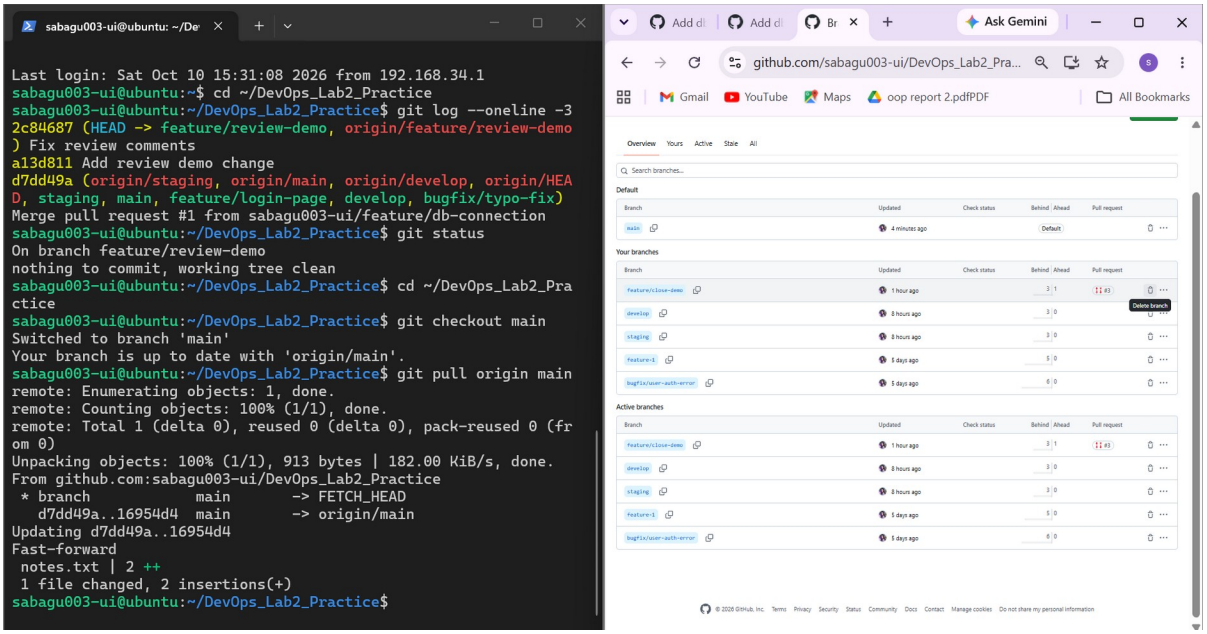


Figure 13.1: Split-screen: remote branch deleted from GitHub. (remote_branch_deleted.png)

Step 2: Deleted another completed remote branch from the terminal.

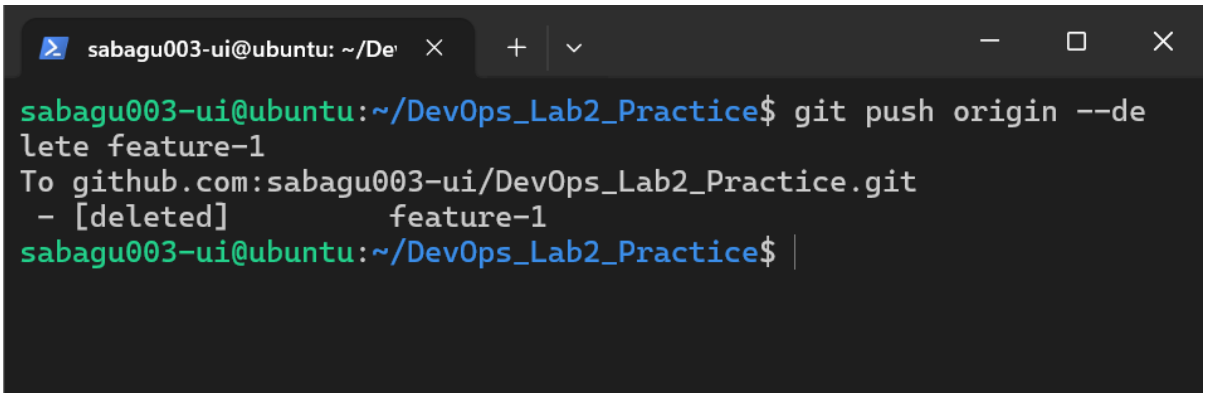


Figure 13.2: Remote branch deleted with git push origin --delete. (remote_branch_delete_cmd.png)

Step 3: Updated the local repository and removed the local branch.

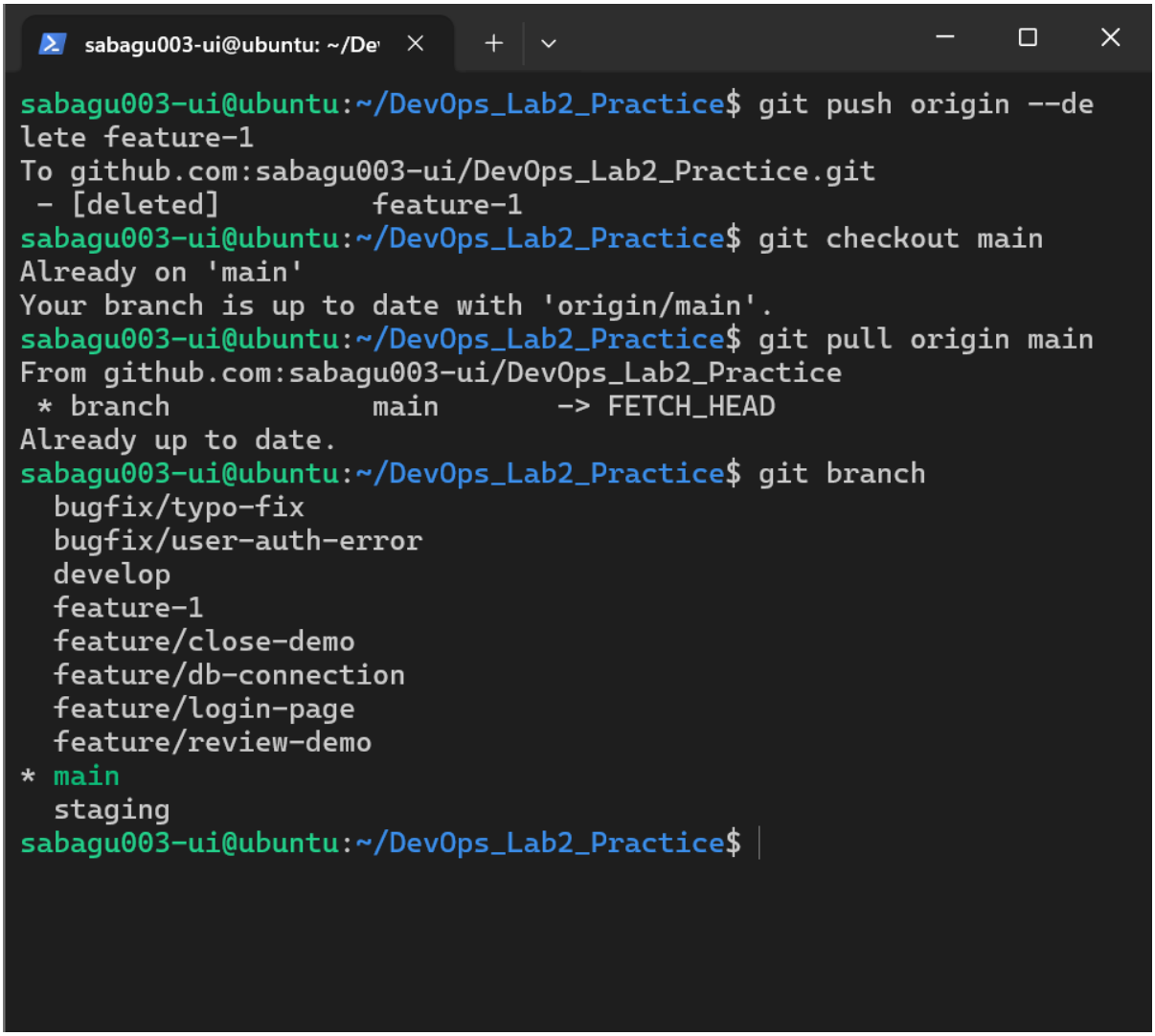


Figure 13.3: Local repository updated and branch removed.

Bonus Task: Simulated Team Collaboration

Step 1: Created the branch collab; both users pulled, added their name and a fun fact to notes.txt, committed and pushed to collab.

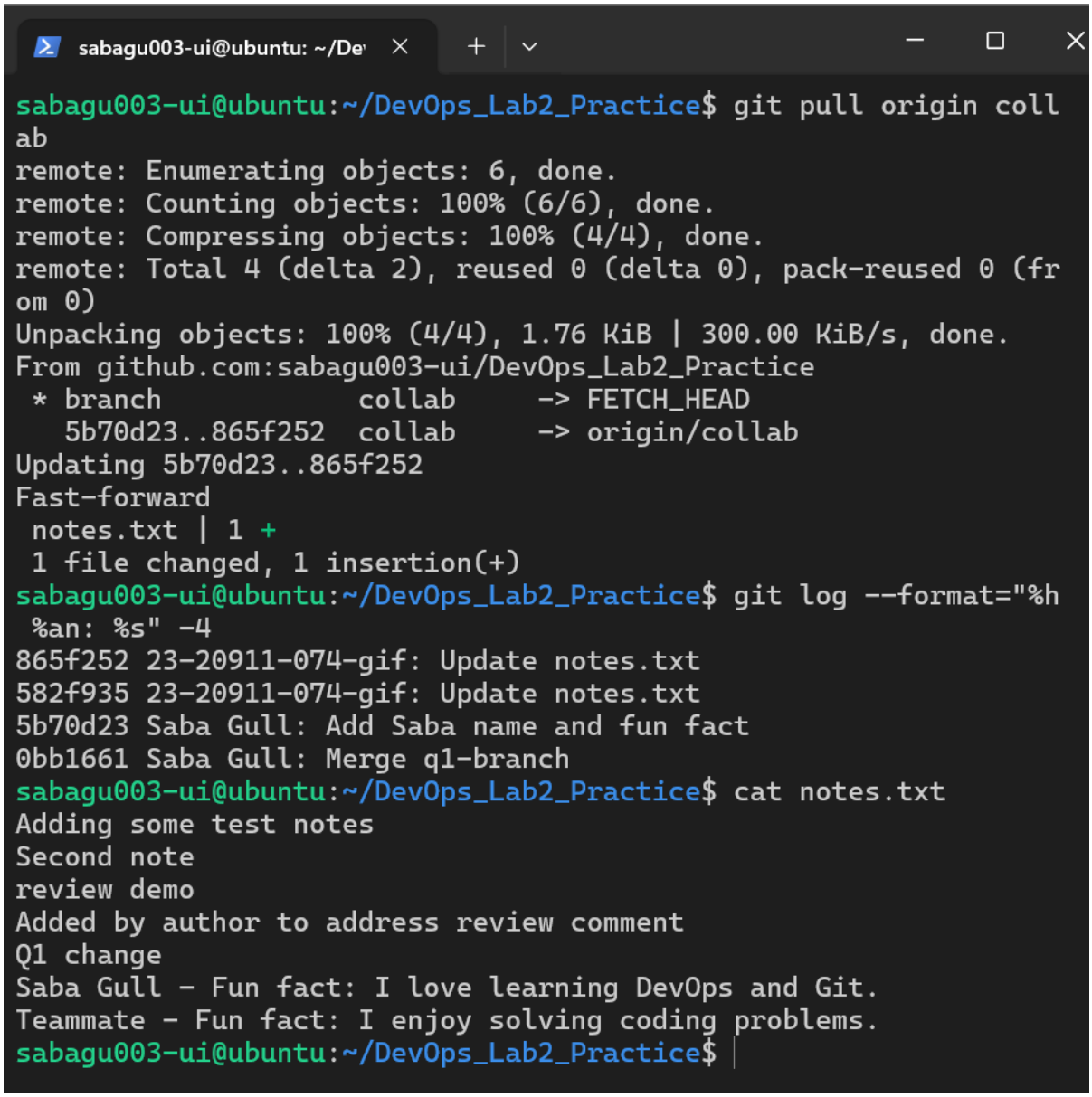


Figure B.1: Collaboration commits on the collab branch. (collab_commit.png)

Step 2: Merged collab into main and pushed.

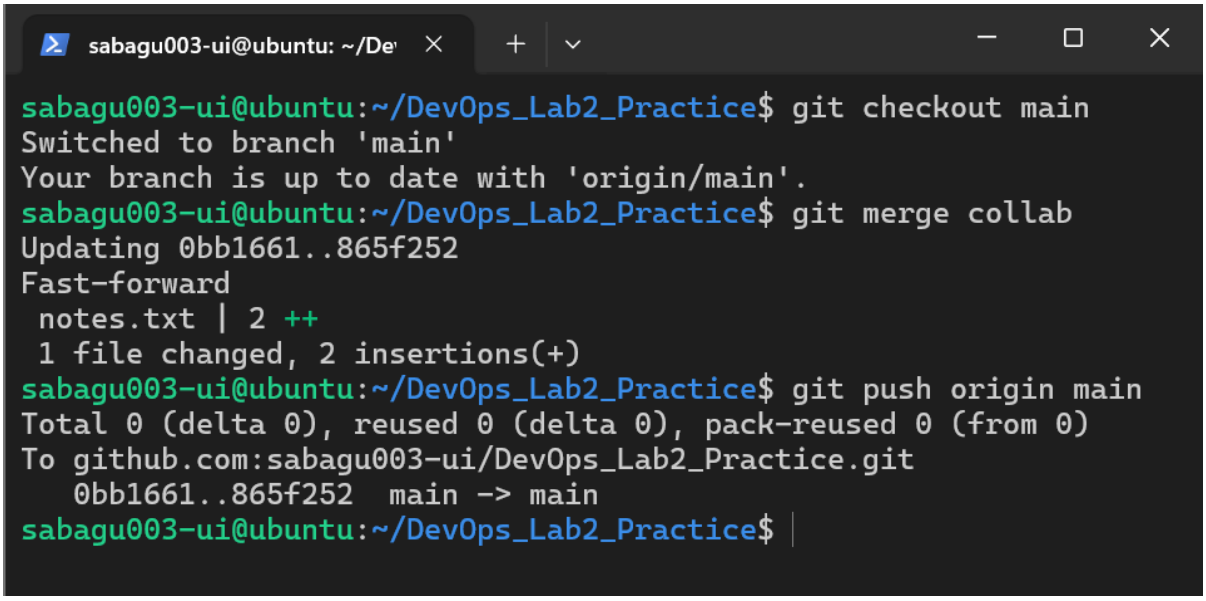


Figure B.2: collab merged into main. (collab_merge.png)

Exam Q1: Advanced Branching & Merge Verification

Step 1: Created a new branch.

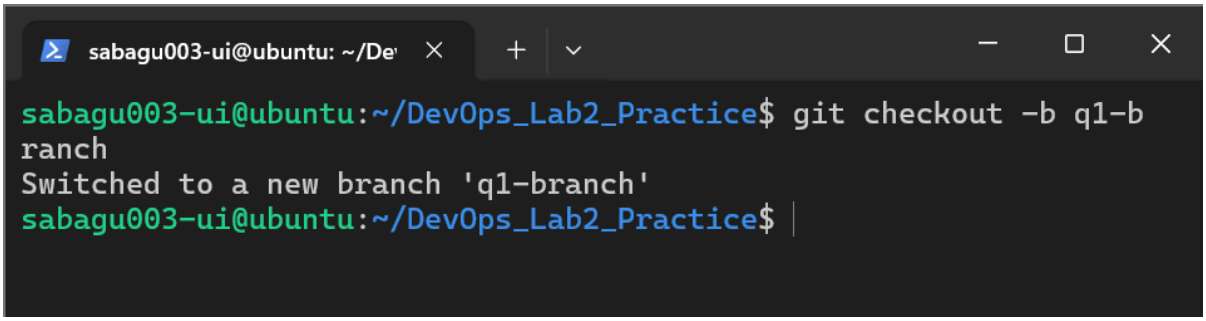


Figure Q1.1: Branch q1-branch created. (Q1_branch_created.png)

Step 2: Made a small change and committed it.

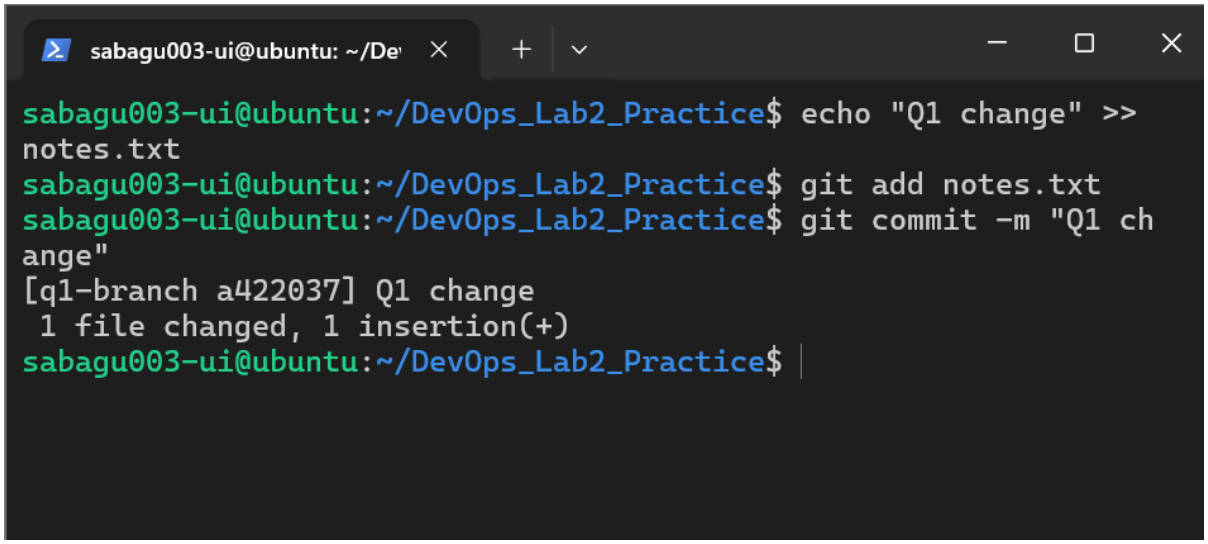


Figure Q1.2: Q1 commit done. (Q1_commit_done.png)

Step 3: Merged the branch back into main.

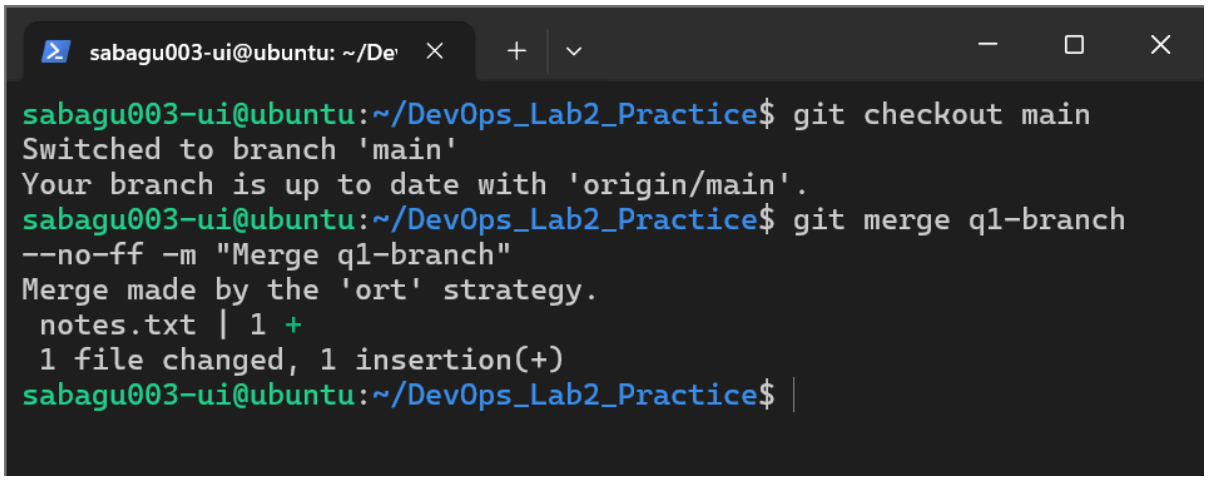


Figure Q1.3: q1-branch merged into main. (Q1_merge_done.png)

Step 4: Showed the commit history that verifies the merge.

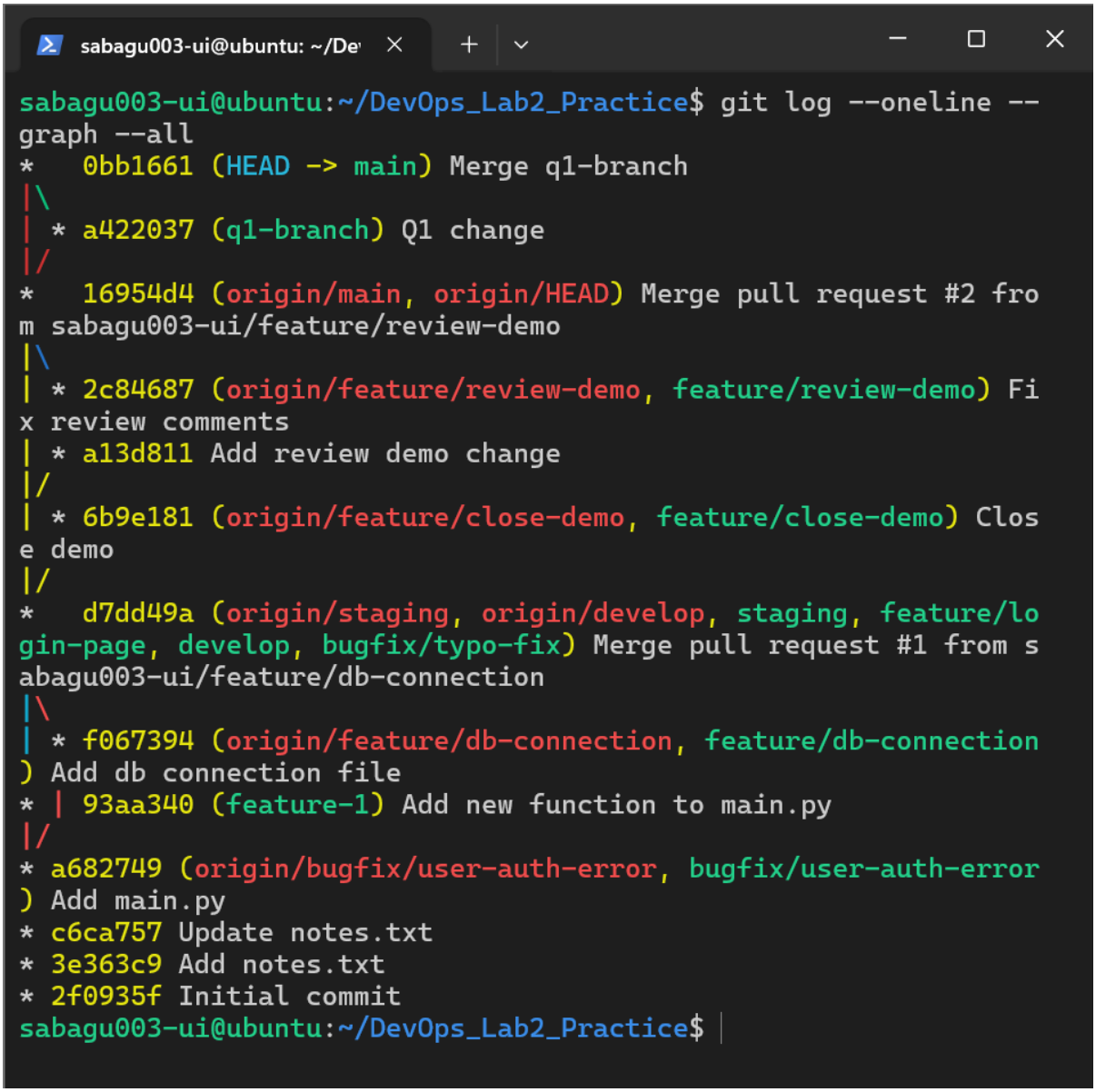


Figure Q1.4: Commit graph showing the merge commit. (Q1_merge_history.png)

Exam Q2: Multi-Stage Workflow Simulation

Step 1: Showed the three workflow branches: main, develop and staging.

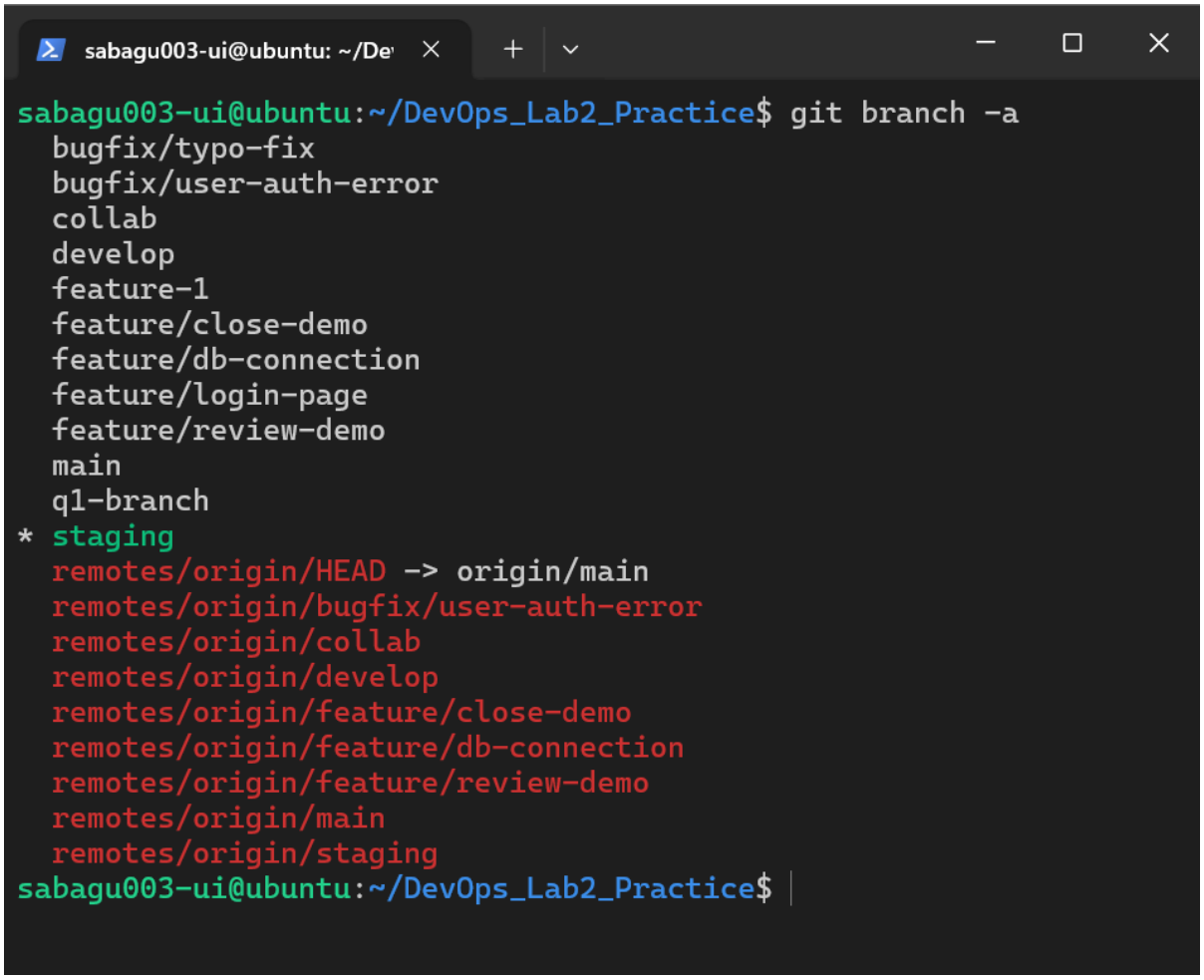


Figure Q2.1: main, develop and staging branches. (Q2_branches_created.png)

Step 2: Created a feature branch from develop, made changes and committed.

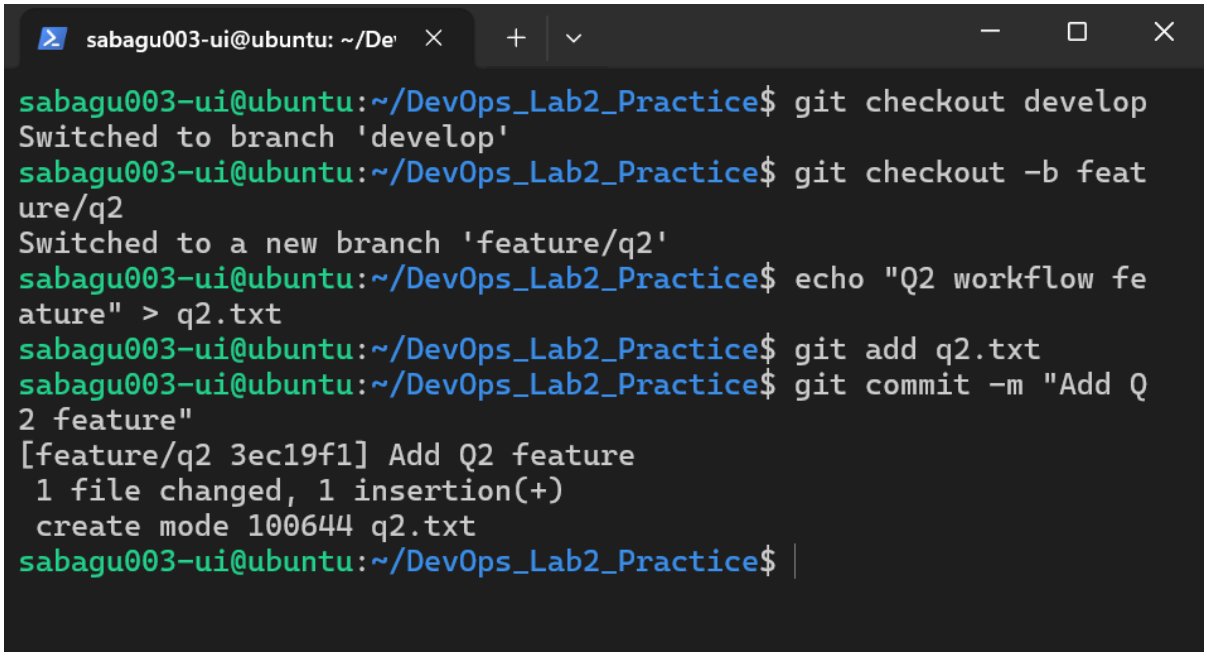


Figure Q2.2: Feature branch commit. (Q2_feature_branch_commit.png)

Step 3: Merged the feature branch into develop.

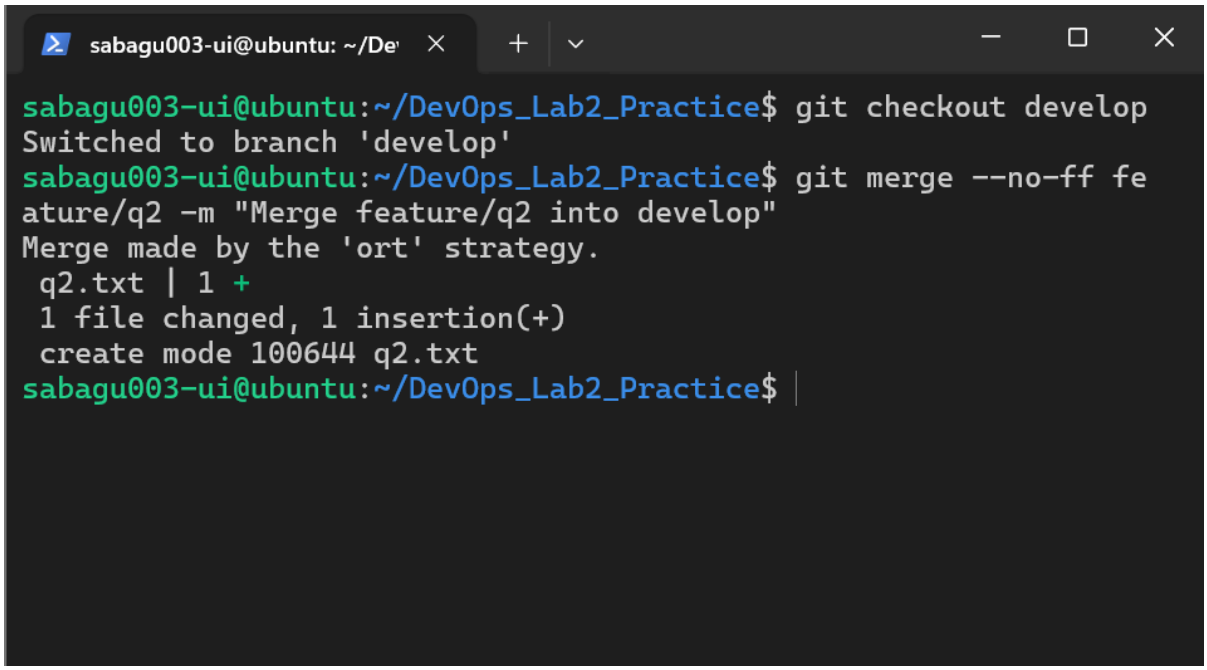


Figure Q2.3: Feature merged into develop. (Q2_merge_into_develop.png)

Step 4: Merged develop into staging.

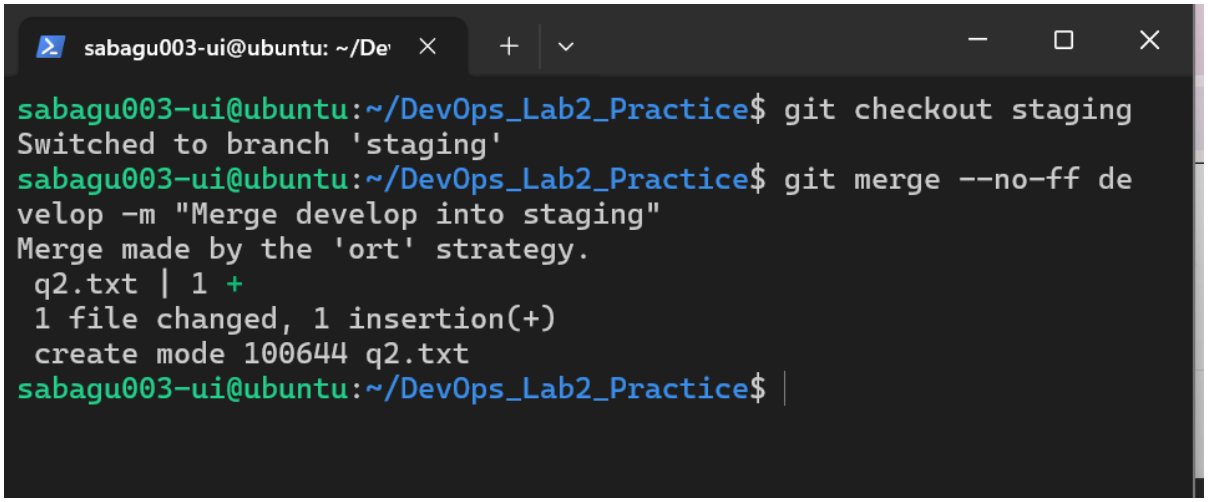


Figure Q2.4: develop merged into staging. (Q2_merge_into_staging.png)

Step 5: Merged staging into main.

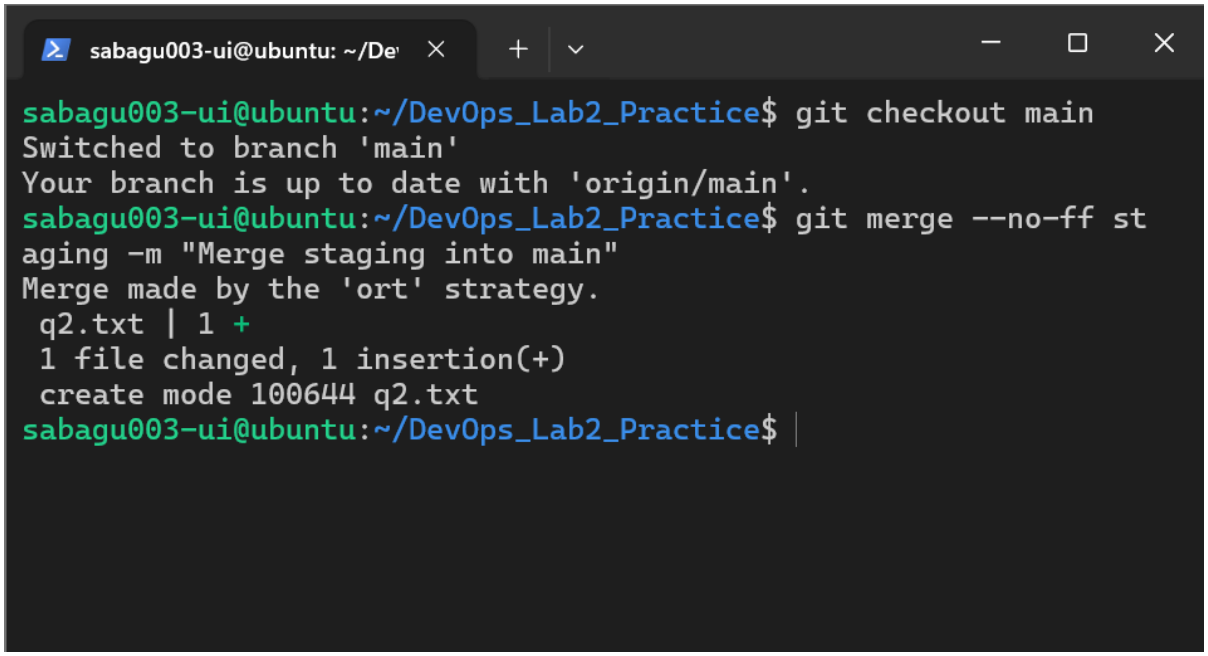


Figure Q2.5: staging merged into main. (Q2_merge_into_main.png)

Step 6: Verified that every stage contains the update.

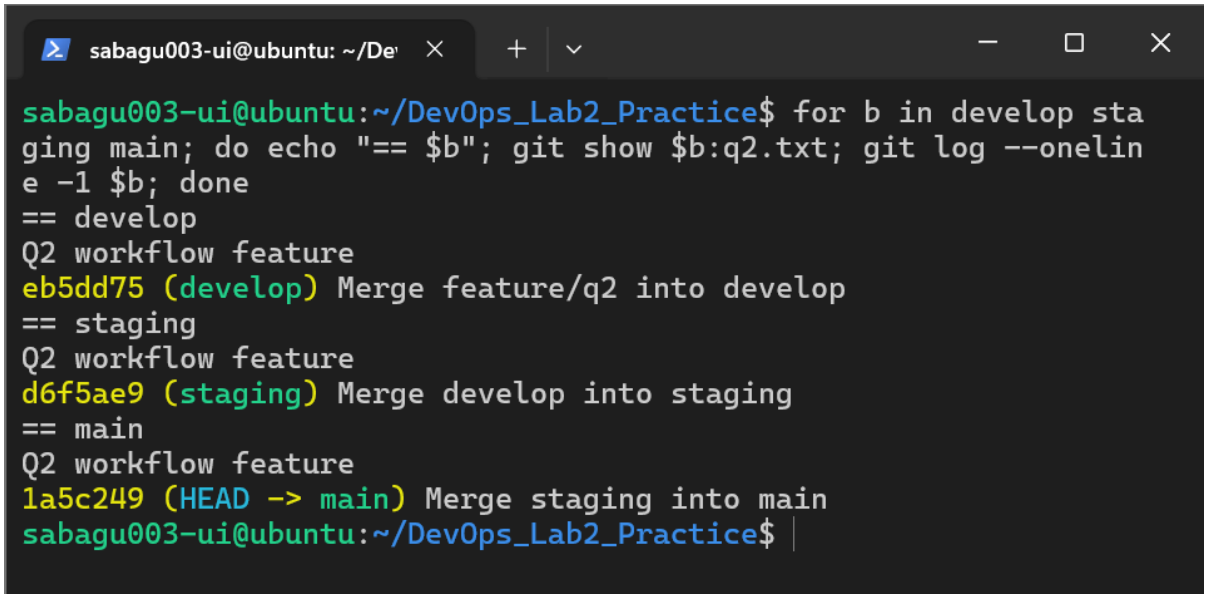


Figure Q2.6: q2.txt and the latest commit present on develop, staging and main. (Q2_stage_verification.png)

Exam Q3: Collaboration & Conflict Resolution

Step 1: Created branch user1 and changed shared.txt.

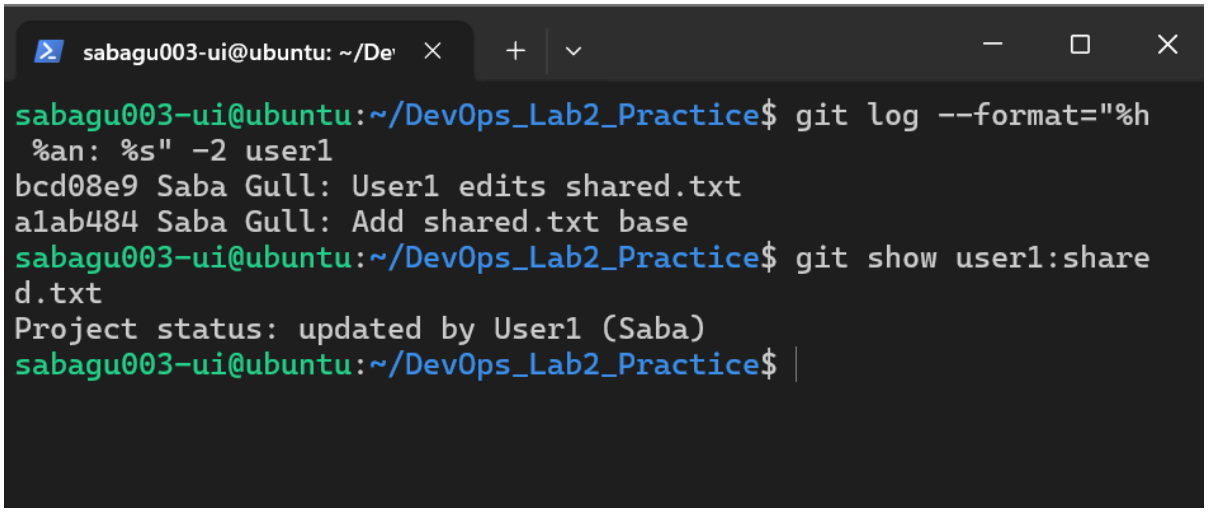


Figure Q3.1: user1 branch modification. (Q3_branches_modified1.png)

Step 2: Created branch user2 and changed the same file.

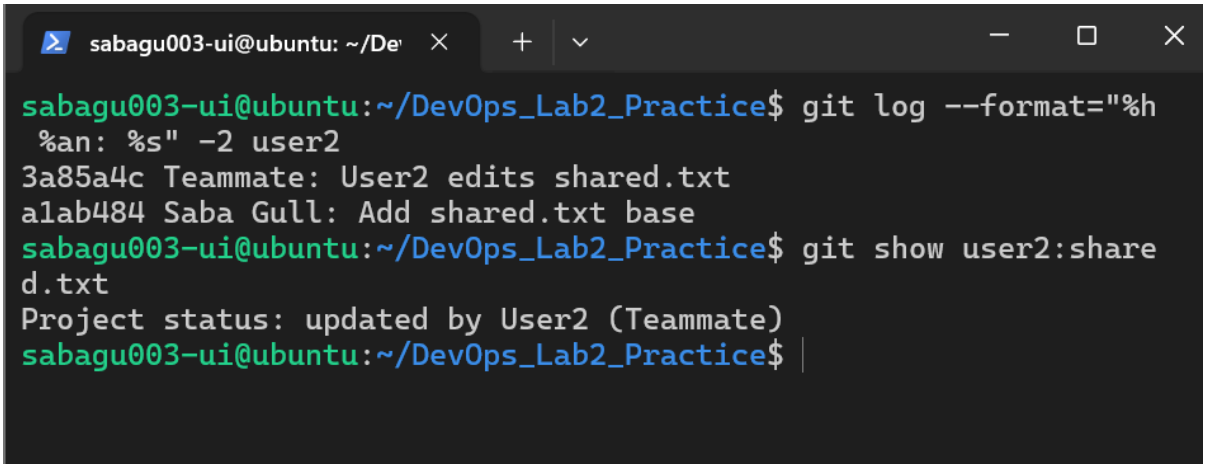


Figure Q3.2: user2 branch modification. (Q3_branches_modified2.png)

Step 3: Showed an overview of both branches.

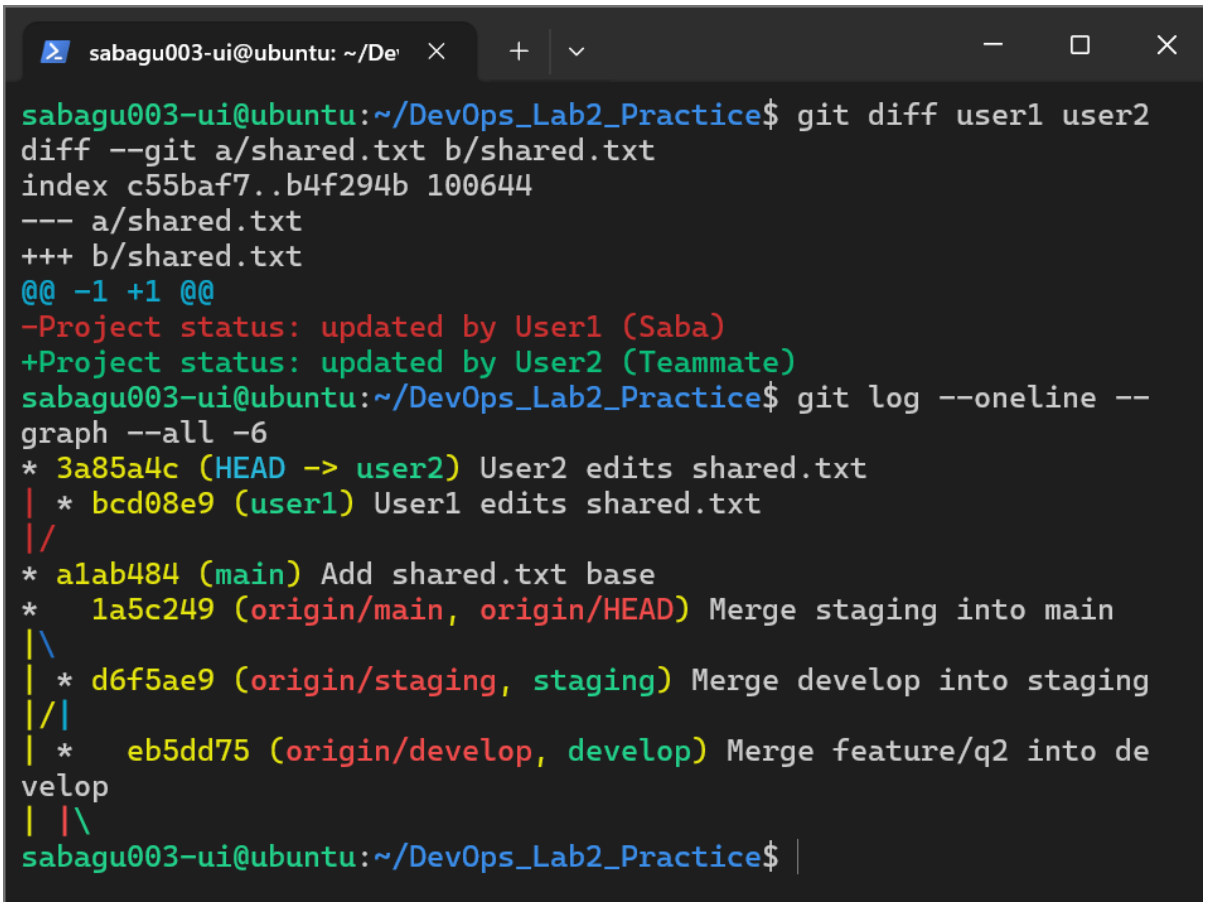


Figure Q3.3: Overview of both branches. (Q3_branches_modified3.png)

Step 4: Merged the branches and captured the conflict.

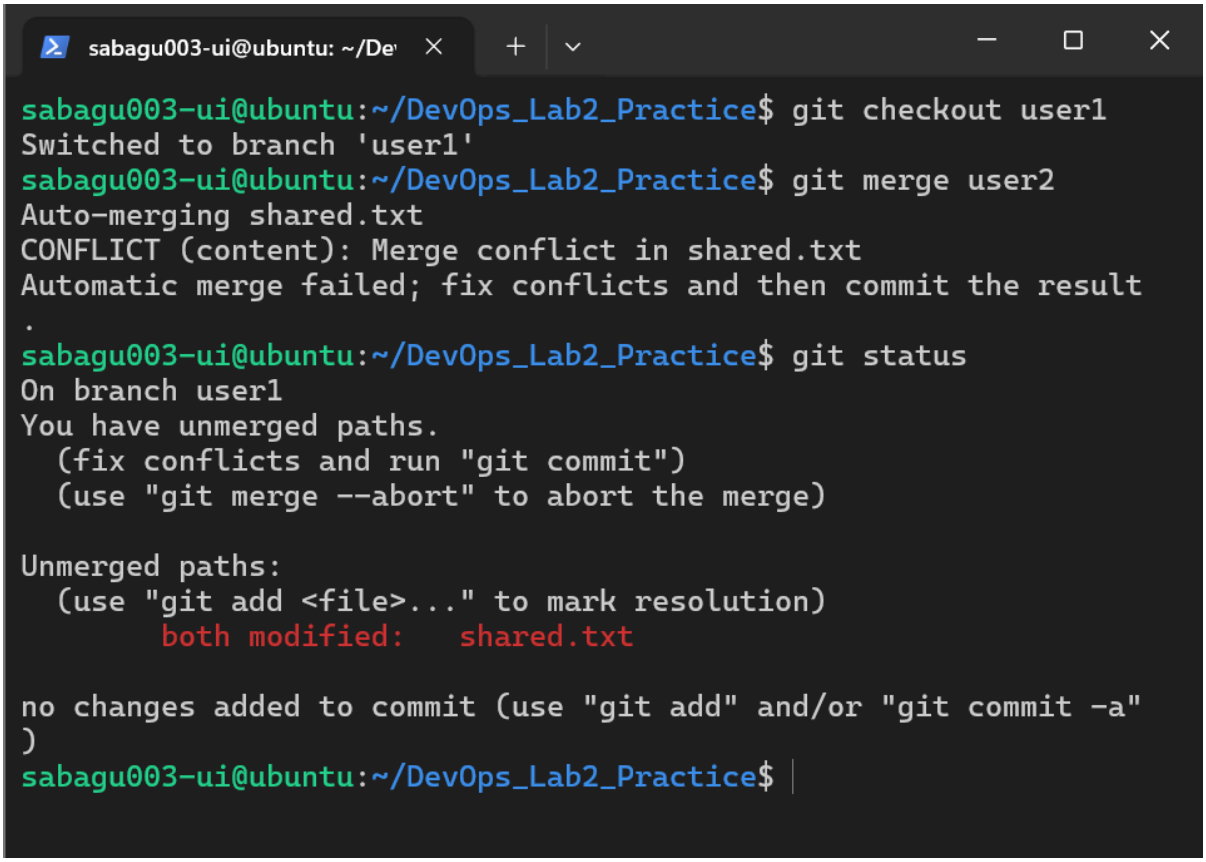


Figure Q3.4: Merge conflict in shared.txt. (Q3_merge_conflict.png)

Step 5: Resolved the conflict by keeping both contributions (removed the conflict markers) and committed.

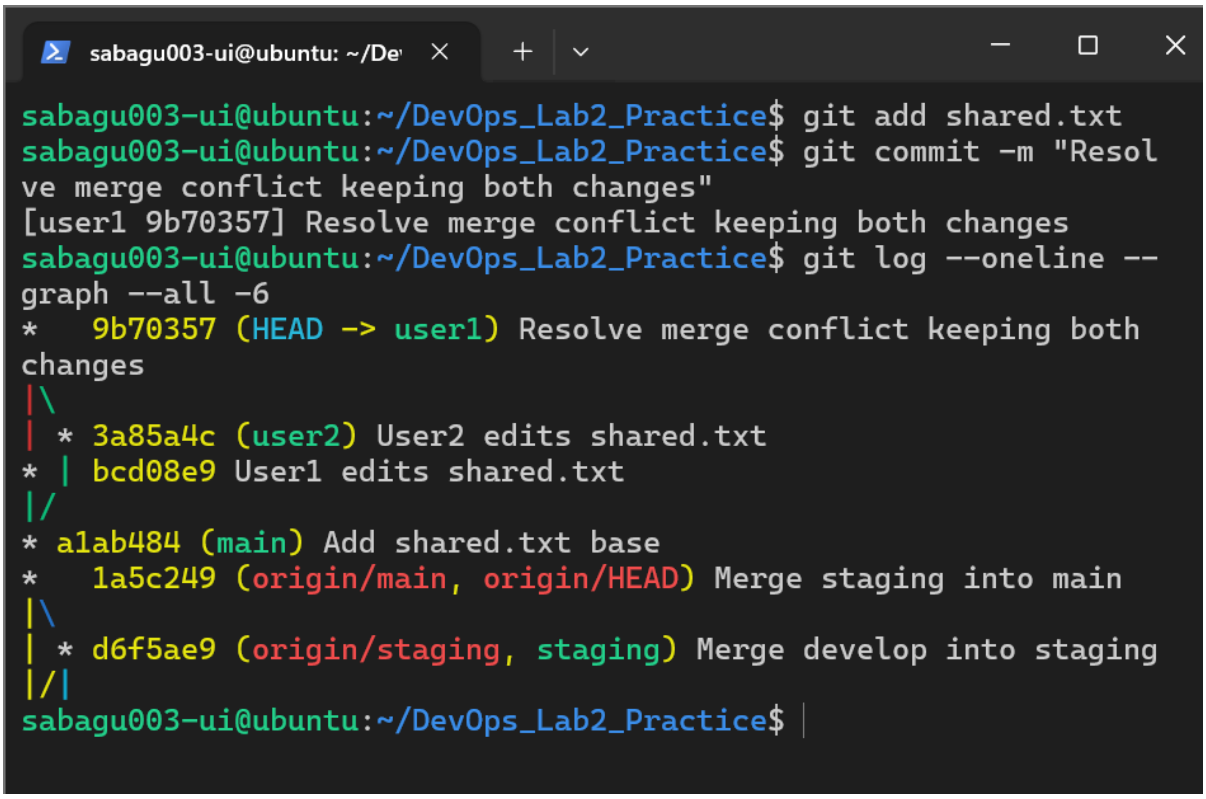


Figure Q3.5: Conflict resolved and committed. (Q3_conflict_resolved.png)

Step 6: Showed the final file containing both collaborators' changes.

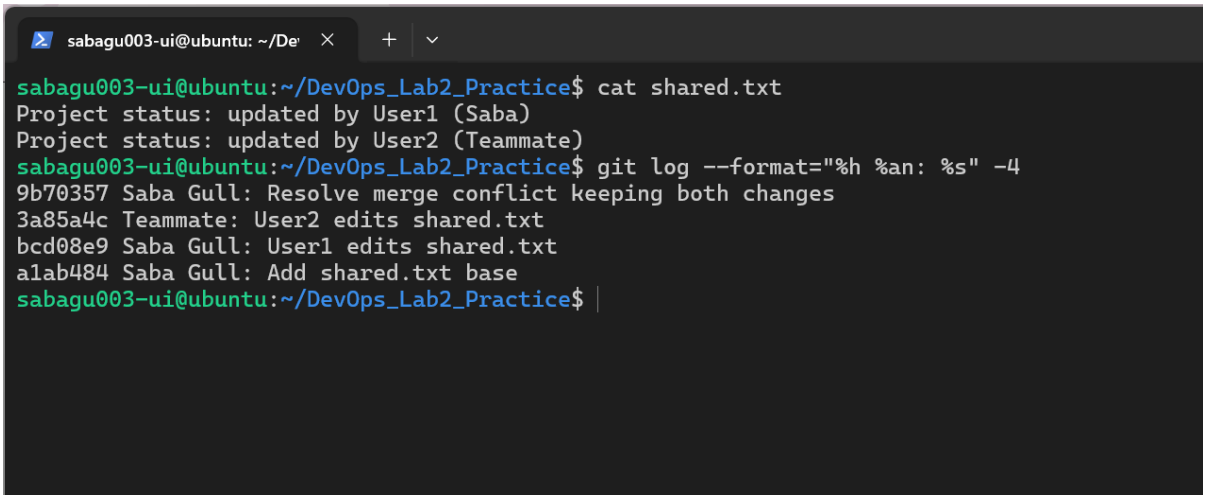


Figure Q3.6: Final shared.txt with both changes. (Q3_final_file.png)